DATA 255 Deep Learning Technologies

Oct 23, 2025

Taehee Jeong, Ph.D.

SJSU SAN JOSÉ STATE UNIVERSITY

Topics(1)

- Object detection

Object detection

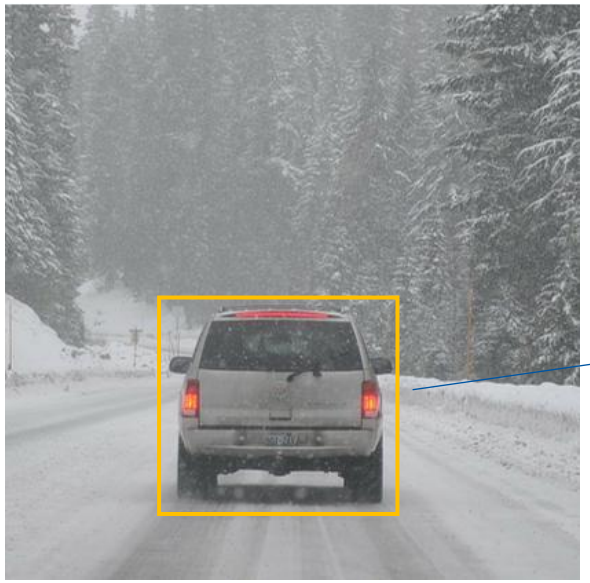
- Object localization
- Bounding boxes
- Sliding window
- 1x1 convolution
- Intersection over union
- Non-max suppression

Object localization

Image: deep learning, Andrew Ng

Where is the car located in the image?

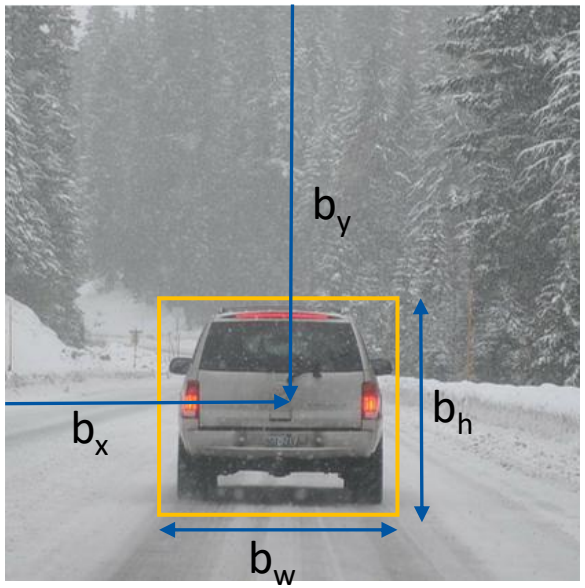
Object localization



Source: deep learning, Andrew Ng

Bounding box

(0,0)

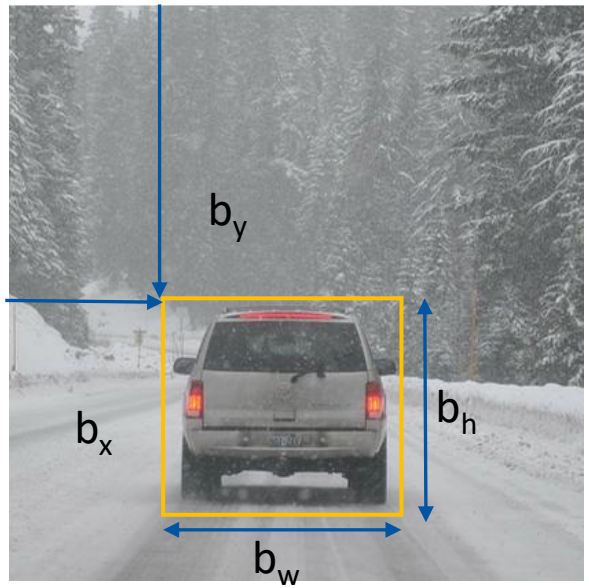


(1,1)

Bounding box
$b_x = 0.5$
$b_y = 0.7$
$b_h = 0.3$
$b_w = 0.4$

Bounding box (another way)

(0,0)

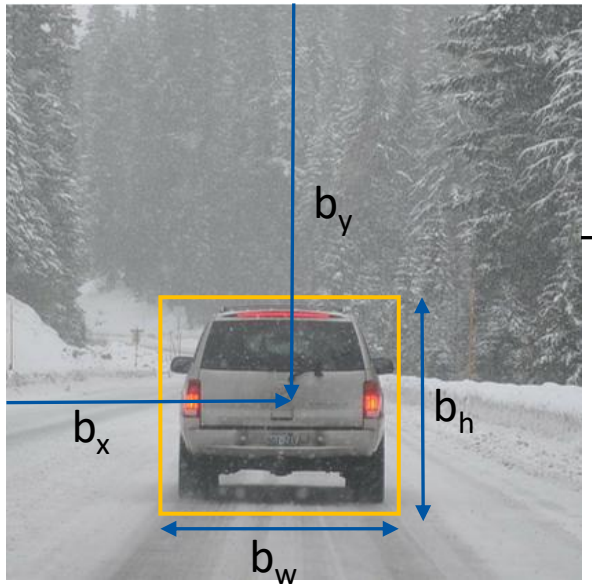


(1,1)

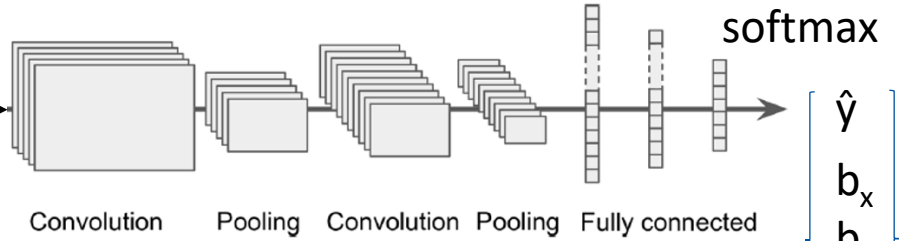
Bounding box
$b_x = 0.3$
$b_y = 0.5$
$b_h = 0.3$
$b_w = 0.4$

Localization with CNN

(0,0)



(1,1)



$\hat{y}$

b_x

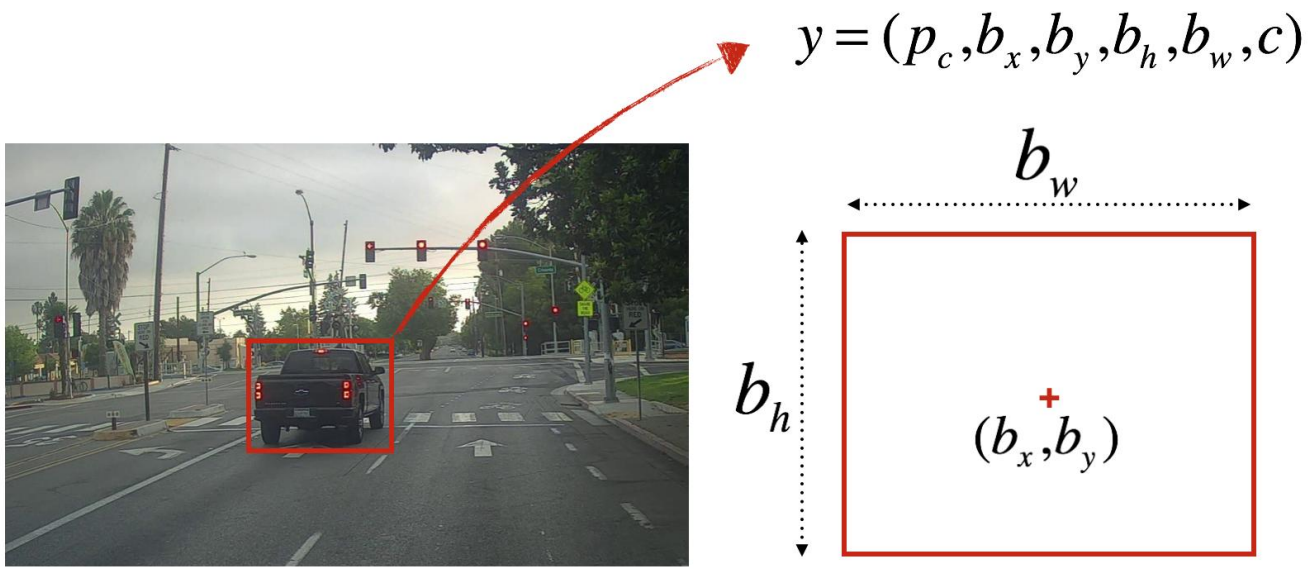
b_y

b_h

b_w

Source: deep learning, Andrew Ng

Example of bounding box



$p_c = 1$: confidence of an object being present in the bounding box

$c = 3$: class of the object being detected (here 3 for “car”)

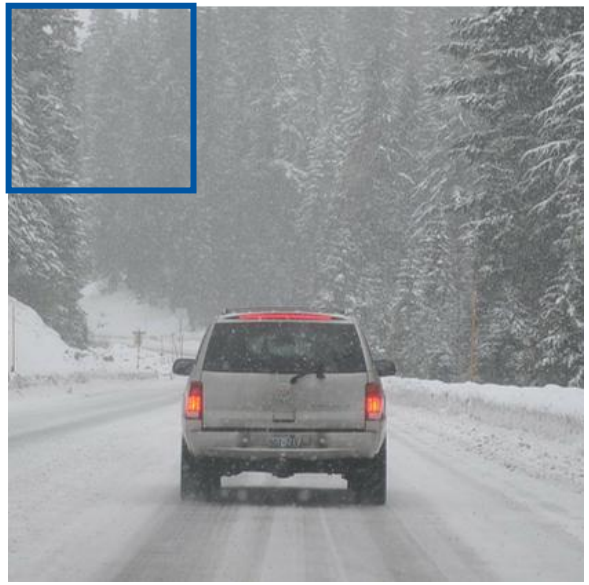
Source: deep learning, Andrew Ng

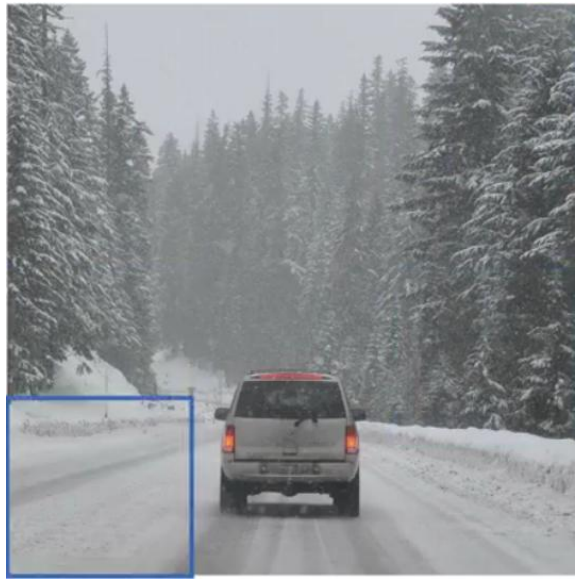
Multiple Objects detection

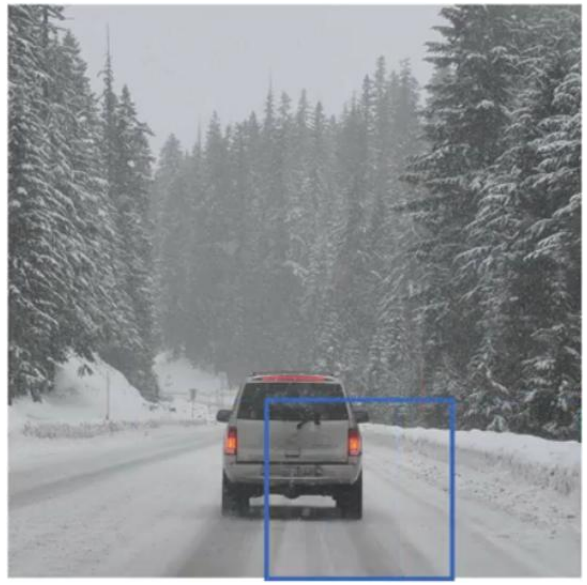
How about multiple objects in one image?

Source: deep learning, Andrew Ng

Sliding windows detection







Source: Deep Learning, Andrew Ng

Sliding windows with various window size

Instead of one size,
Many different size of windows (from small one to larger one) are needed to apply.

Source: Deep Learning, Andrew Ng

Issue of sliding windows

Implementation Problem :

sliding windows detection takes long time to compute.

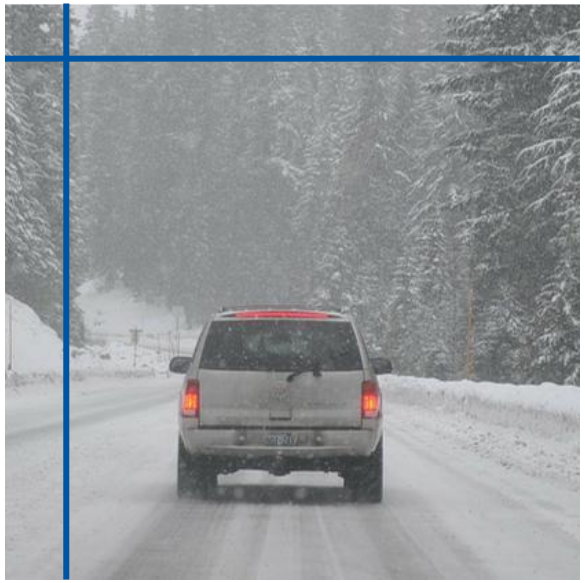
Issue of sliding windows

Input image size: 256x256

Sliding window: 32x32

Stride: 32

Need to move & compute 8x8 times!



Convolution implementation of sliding windows

Sliding windows is similar operation of convolution.

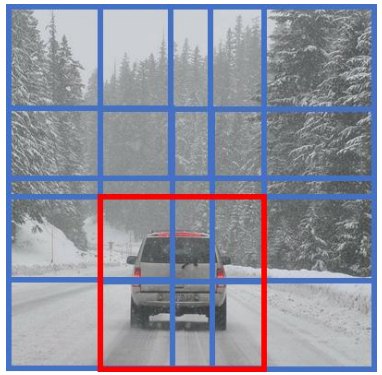
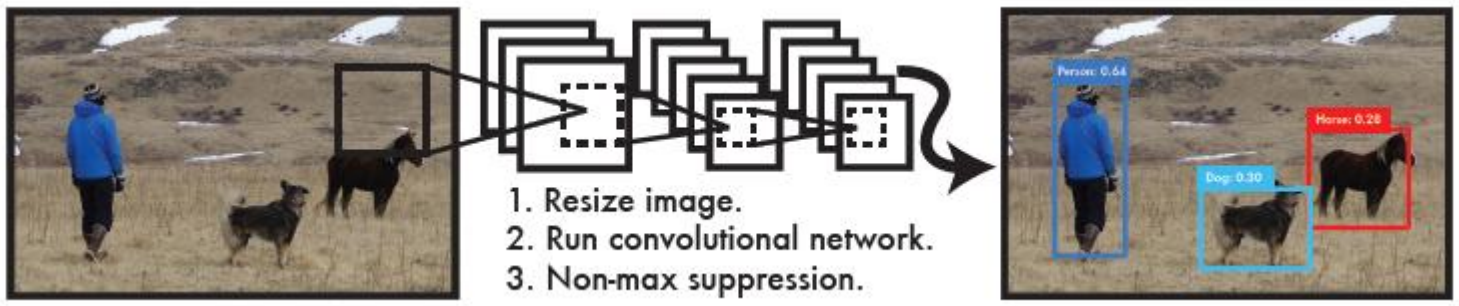


Image: Deep Learning, Andrew Ng

Source: OverFeat: Integrated recognition, localization and detection using convolutional networks, Sermanet et al., 2014,

You Only Look Once: Unified real-time object detection, Redmon et al., 2015

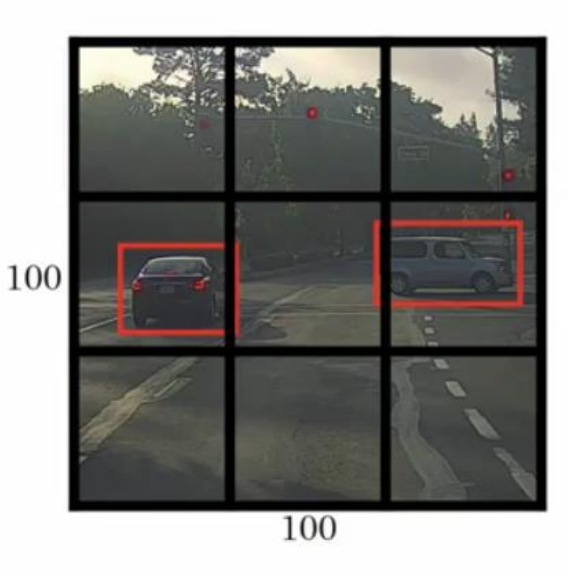
YOLO (you only look once)



(1) Resizes the input image
(2) runs a single convolutional network on the image
(3) thresholds the resulting detections by the model's confidence.

You Only Look Once: Unified real-time object detection, Redmon et al., 2015

Divide image into S x S grid



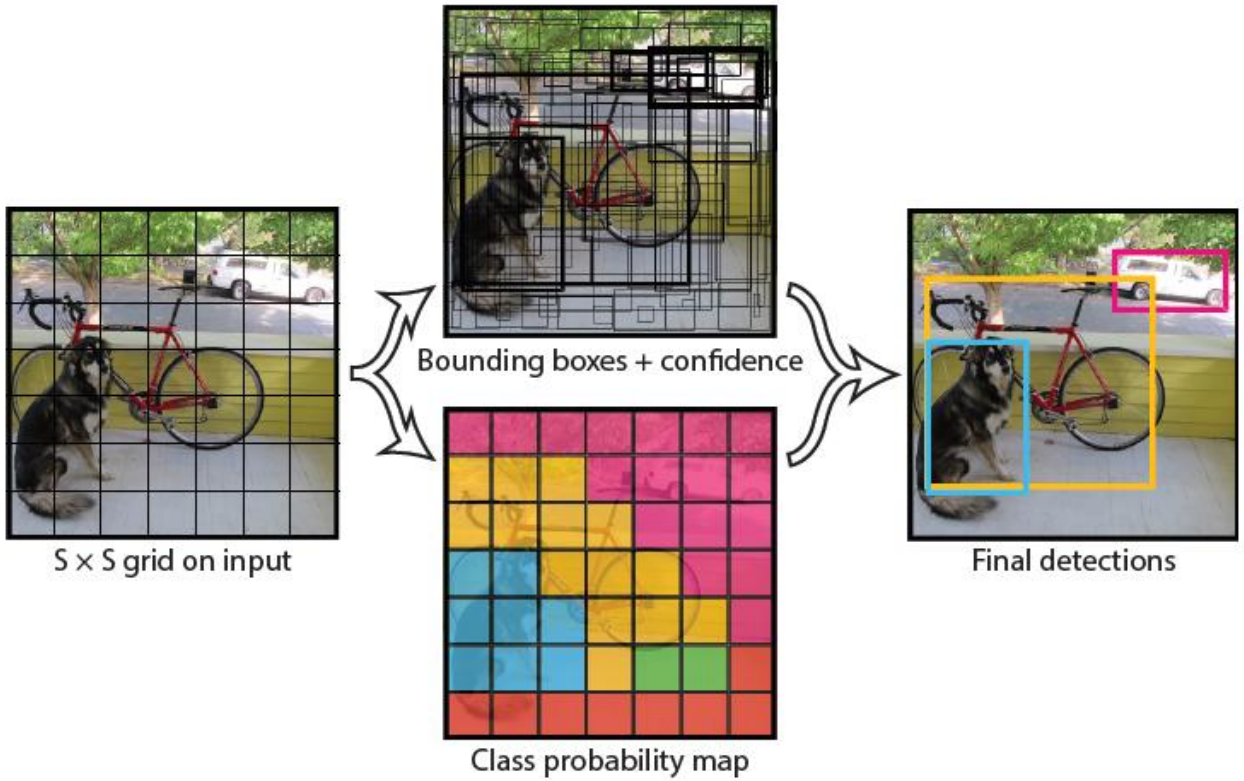
$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

For each grid cell:

- Bounding box
- Confidence for those boxes,
- C class probabilities

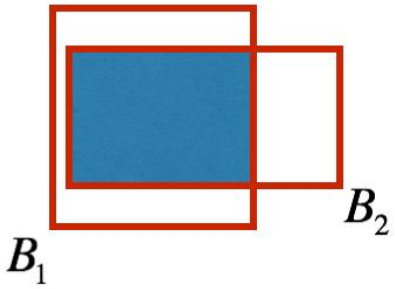
You Only Look Once: Unified real-time object detection, Redmon et al., 2015

YOLO

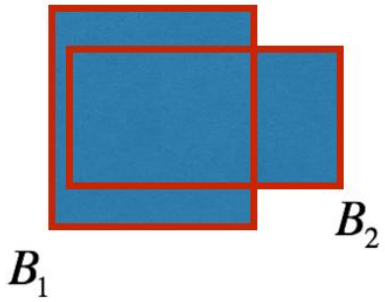


Intersection over Union (IoU)

Intersection

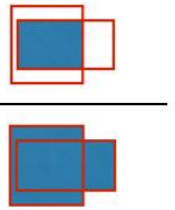


Union



Intersection over Union

$$IoU = \frac{B_1 \cap B_2}{B_1 \cup B_2} =$$



$=P_c$

"Correct" if IoU $\geq$ 0.5

More generally, IoU is a measure of the overlap between two bounding boxes.

Source: Deep Learning, Andrew Ng

Confusion matrix

		Predicted label	
		1	0
True label	1	True positive (tp)	False negative (fn)
	0	False positive (fp)	True negative (tn)

$$Accuracy = \frac{tp + tn}{tp + fn + fp + tn}$$

$$Precision = \frac{tp}{tp + fp}$$

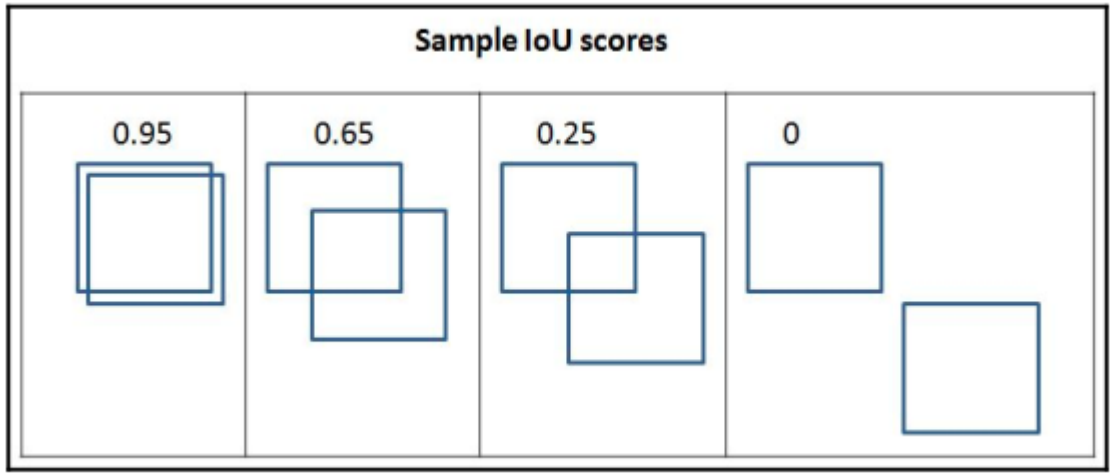
$$Recall = \frac{tp}{tp + fn}$$

Precision

TP if IoU ≥ 0.5

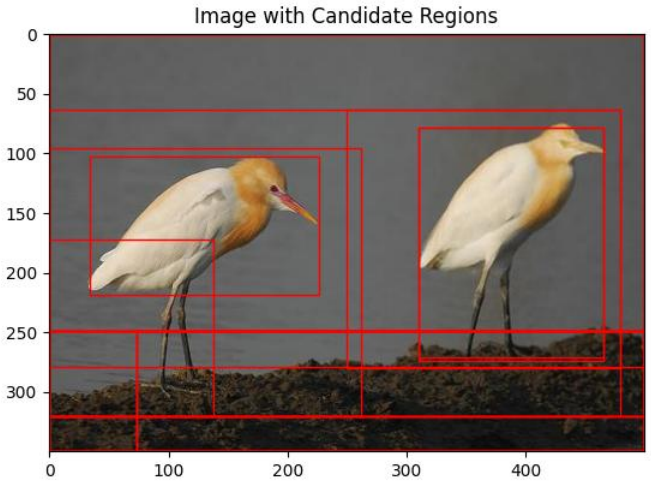
FP if IoU < 0.5

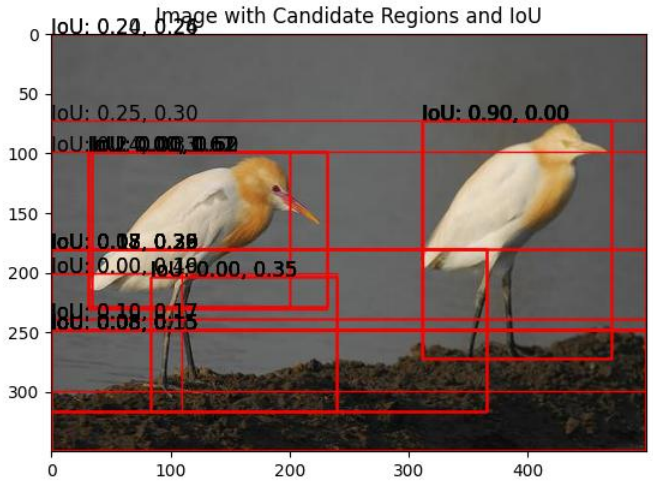
$$Precision = \frac{tp}{tp + fp}$$

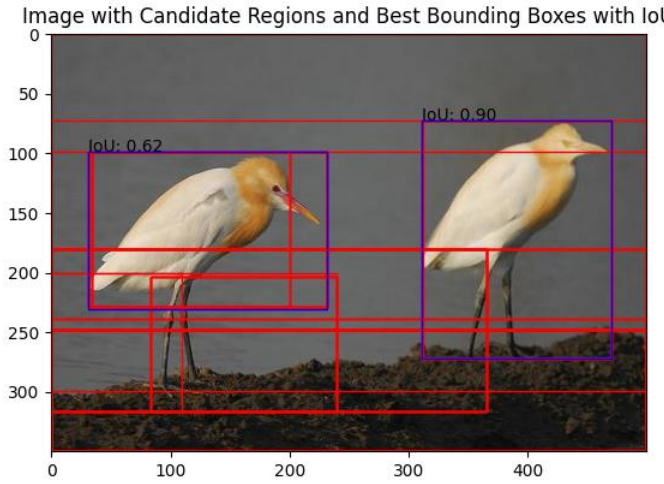


Source: https://www.analyticsvidhya.com/blog/2024/02/evaluation-matrix-for-object-detection-using-iou-and-map/

Non-max suppression







Source: https://www.analyticsvidhya.com/blog/2024/02/evaluation-matrix-for-object-detection-using-iou-and-map/

Non-max suppression

Each output prediction is:

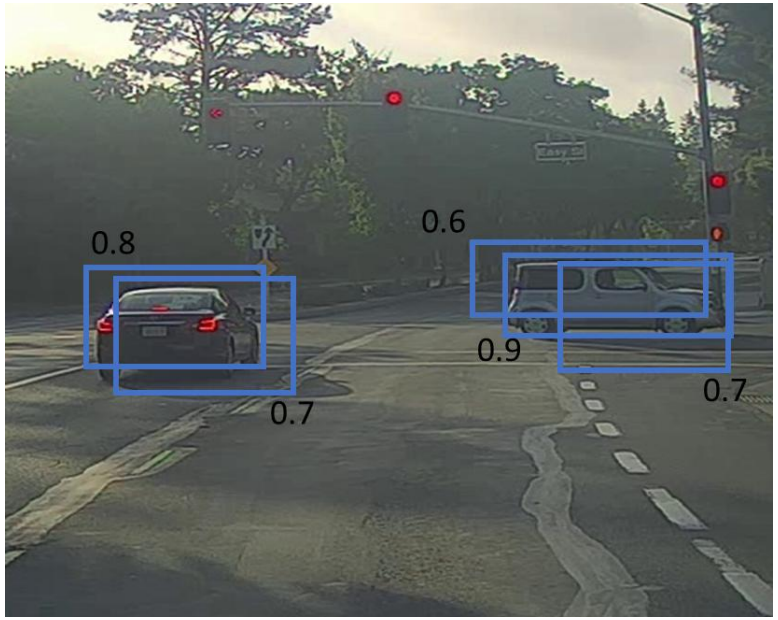
$$\begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \end{bmatrix}$$

- Discard all boxes with $p_c \leq 0.6$
- Among remaining boxes,
- Pick the box with the largest p_c.
- Output that as a prediction.

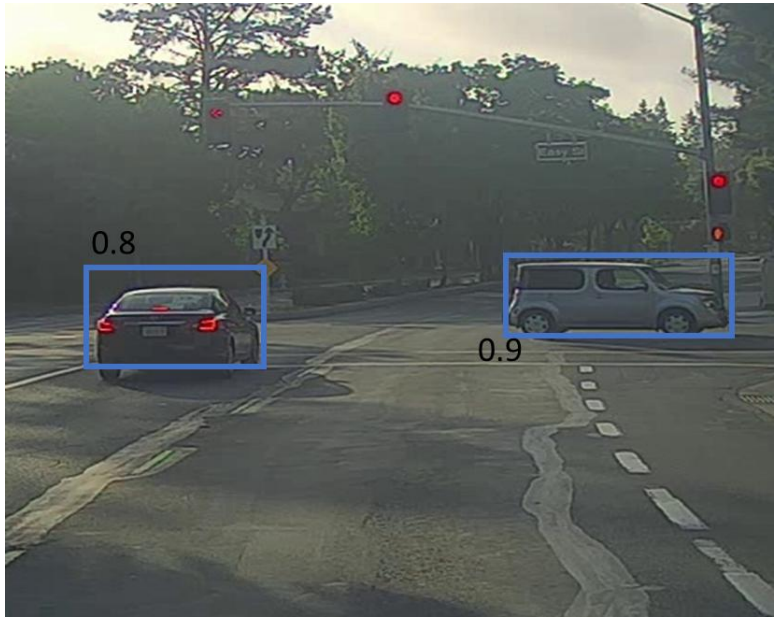
Source: Deep Learning, Andrew Ng

Non-max suppression

Before non-max suppression



After non-max suppression



Source: Deep Learning, Andrew Ng

Average Precision (AP)

Average Precision (AP):

AP	% AP at IoU=.50:.05:.95 (primary challenge metric)
$AP^{IoU=.50}$	% AP at IoU=.50 (PASCAL VOC metric)
$AP^{IoU=.75}$	% AP at IoU=.75 (strict metric)

AP Across Scales:

AP^{small}	% AP for small objects: area < 32^2
AP^{medium}	% AP for medium objects: 32^2 < area < 96^2
AP^{large}	% AP for large objects: area > 96^2

	backbone	AP	AP_{50}	AP_{75}	AP_S	AP_M	AP_L
Two-stage methods							
Faster R-CNN+++ [5]	ResNet-101-C4	34.9	55.7	37.4	15.6	38.7	50.9
Faster R-CNN w FPN [8]	ResNet-101-FPN	36.2	59.1	39.0	18.2	39.0	48.2
Faster R-CNN by G-RMI [6]	Inception-ResNet-v2 [21]	34.7	55.5	36.7	13.5	38.1	52.0
Faster R-CNN w TDM [20]	Inception-ResNet-v2-TDM	36.8	57.7	39.2	16.2	39.8	**52.1**

Source: https://cocodataset.org/#detection-eval & arxiv:1804.02767

Mean Average Precision (mAP)

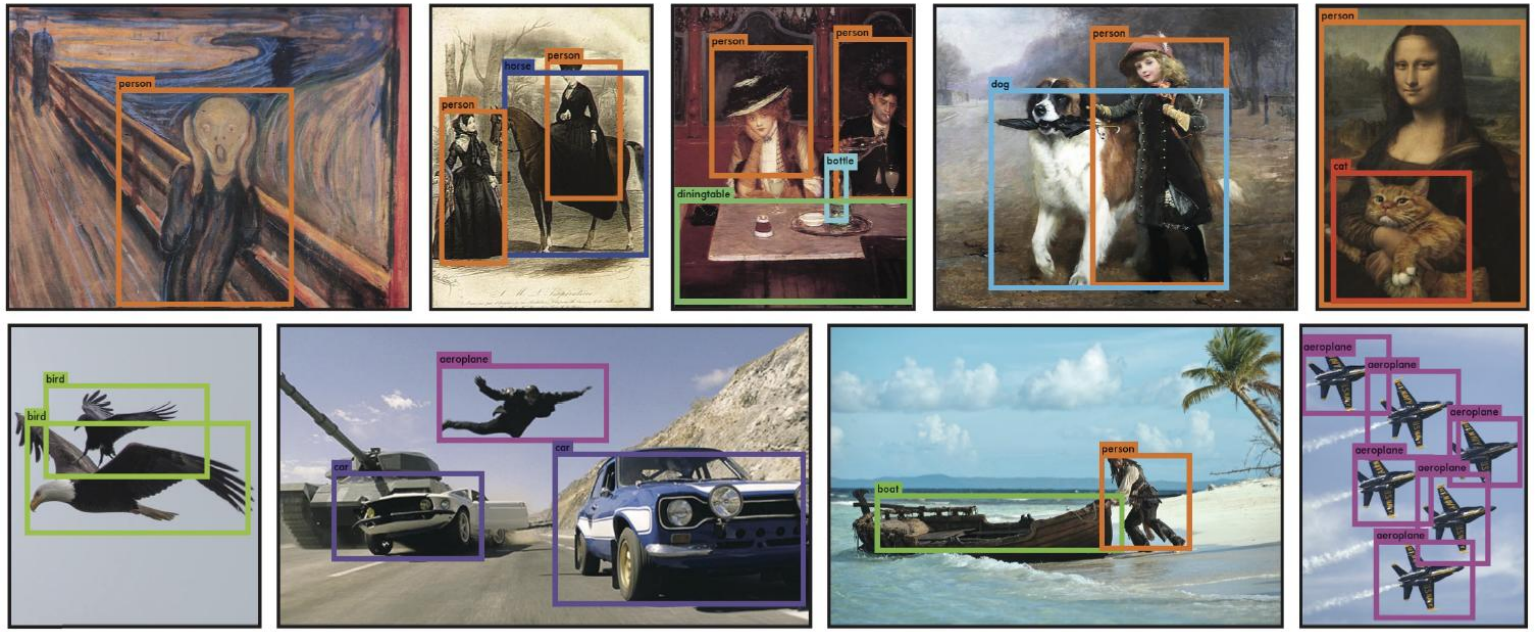
- Mean Average Precision (mAP)

Mean Average Precision calculates the average precision across multiple classes and then takes the mean of those average precisions

$$mAP = \frac{1}{N}\sum_{i=1}^{N}(AP_i)$$

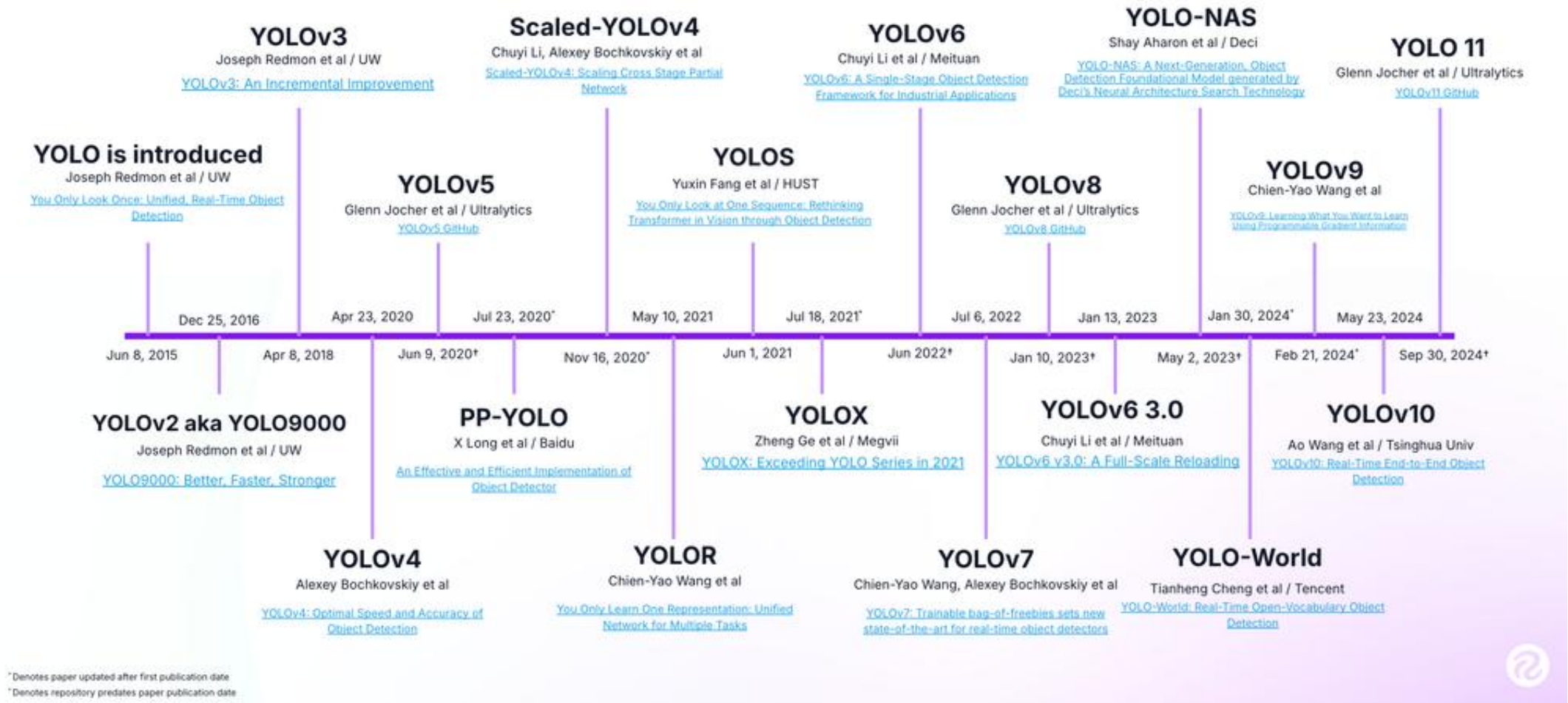
Source: https://cocodataset.org/#detection-eval

Results for YOLO



You Only Look Once: Unified real-time object detection, Redmon et al., 2015

YOLO Greatest Hits



Source: https://blog.roboflow.com/guide-to-yolo-models/

Agricultural Applications of YOLO

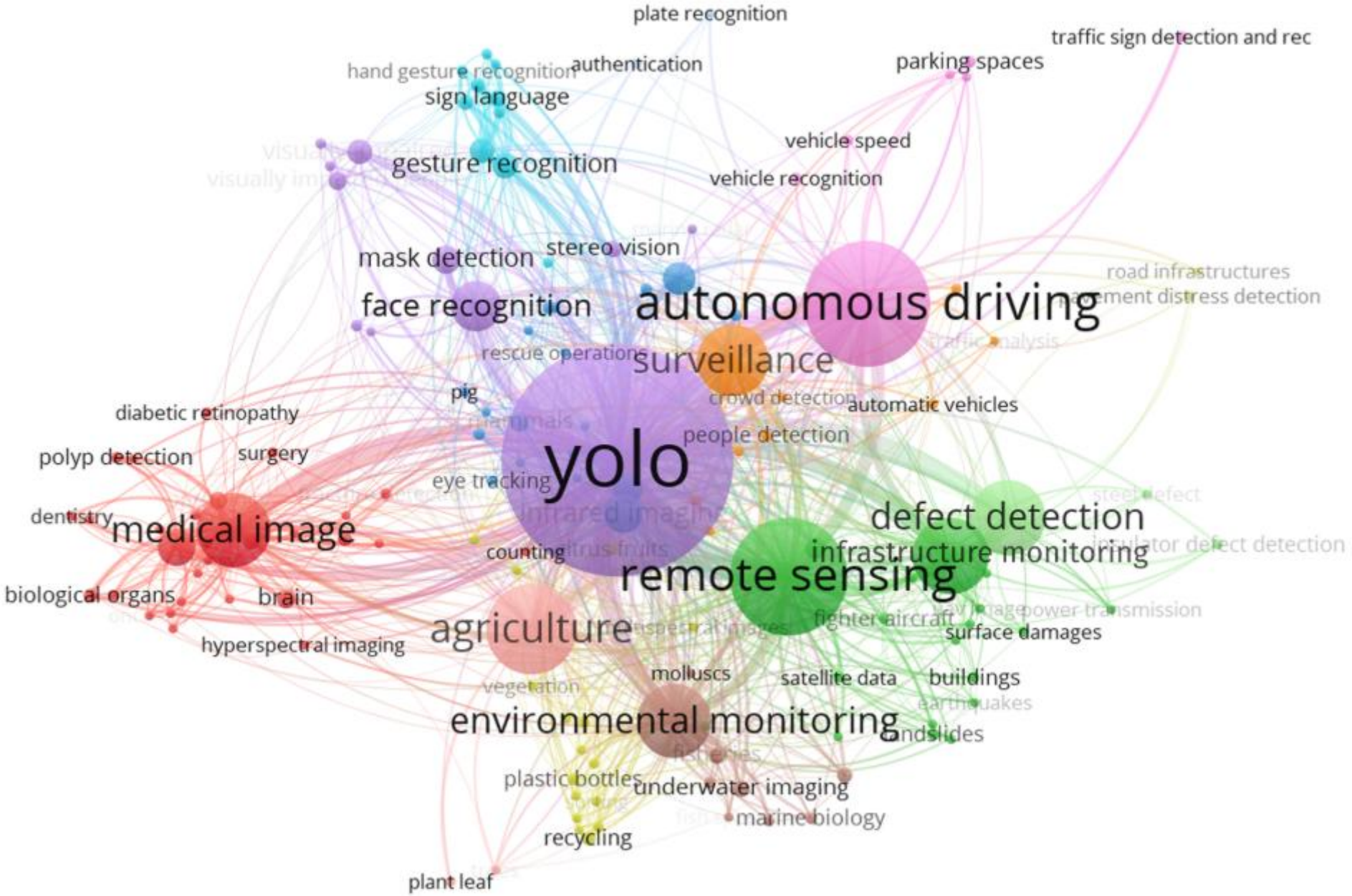
- Weed Detection
- Crop Detection
- Animal Tracking
- Disease Detection in Agriculture
- Smart Farming (disease and pest detection, seed classification)

YOLO Versions Across Various Application

YOLO Version	Application Domain	Strengths	Limitations
YOLOv1	Object Detection, Weed and Crop Detection [63, 68, 69, 76, 87, 89, 92, 93]	Fast real-time processing	Struggles with small objects
YOLOv2	Animal Tracking [81]	Improved recall, handles small objects better	Higher computational requirements
YOLOv3	Agriculture [61, 70, 71, 72, 77, 86, 88, 92]	Multi-scale detection, good in diverse conditions	Not optimized for very low-power devices
YOLOv4	Agriculture [62, 66, 75, 78, 91, 85]	Robust in complex visual environments	Setup complexity for training on custom datasets
YOLOv5	Weed Detection [65, 67, 94, 95, 84]	Very fast, suitable for real-time applications	May require fine-tuning for specific scenarios
YOLOv6	Wildlife Monitoring [101]	High accuracy with enhanced depth	Needs high computation power for best results
YOLOv7	Crop Disease Detection [96]	High precision, effective in crowded scenes	Computational efficiency decreases with scale
YOLOv8	Agriculture [80, 102]	Very high speed, suitable for dynamic environments	Can struggle with very small or fast-moving objects
YOLOv9	Plant Disease detection [103, 104]	High accuracy and recall, suitable for detailed medical scans	Requires extensive dataset for training
YOLOv10	—	Enhanced accuracy-efficiency trade-off, NMS-free model	Complex configuration for multi-source integration

Source: YOLOv1 to YOLOv10: A comprehensive review of YOLO variants and their application in the agricultural domain

A Decade of You Only Look Once (YOLO) for Object Detection, arxiv.2504.18586



SAN JOSÉ STATE UNIVERSITY

YOLO Variant Comparison

Version	Date	Contributions	Framework
v1	2015	One-shot object detector	Darknet
v2	2016	Multi-scale training, dimensional clustering	Darknet
v3	2018	SPP block, Darknet-53	Darknet
v4	2020	Mish-based activation, CSPDarknet-53 backbone	Darknet
v5	2020	Anchor-free detection, SWISH-based activation, PANet	PyTorch
v6	2022	Self-attention, anchor-free object detection	PyTorch
v7	2022	Transformers, E-ELAN reparameterization	PyTorch
v8	2023	GANs, anchor-free detections	PyTorch
v9	2024	Programmable Gradient Information (PGI), Generalized Efficient Layer Aggregation Network (GELAN)	PyTorch
v10	2024	NMS-free training approach, dual label assignments, holistic model design for enhanced accuracy and efficiency	PyTorch

Source: YOLOv1 to YOLOv10: A comprehensive review of YOLO variants and their application in the agricultural domain

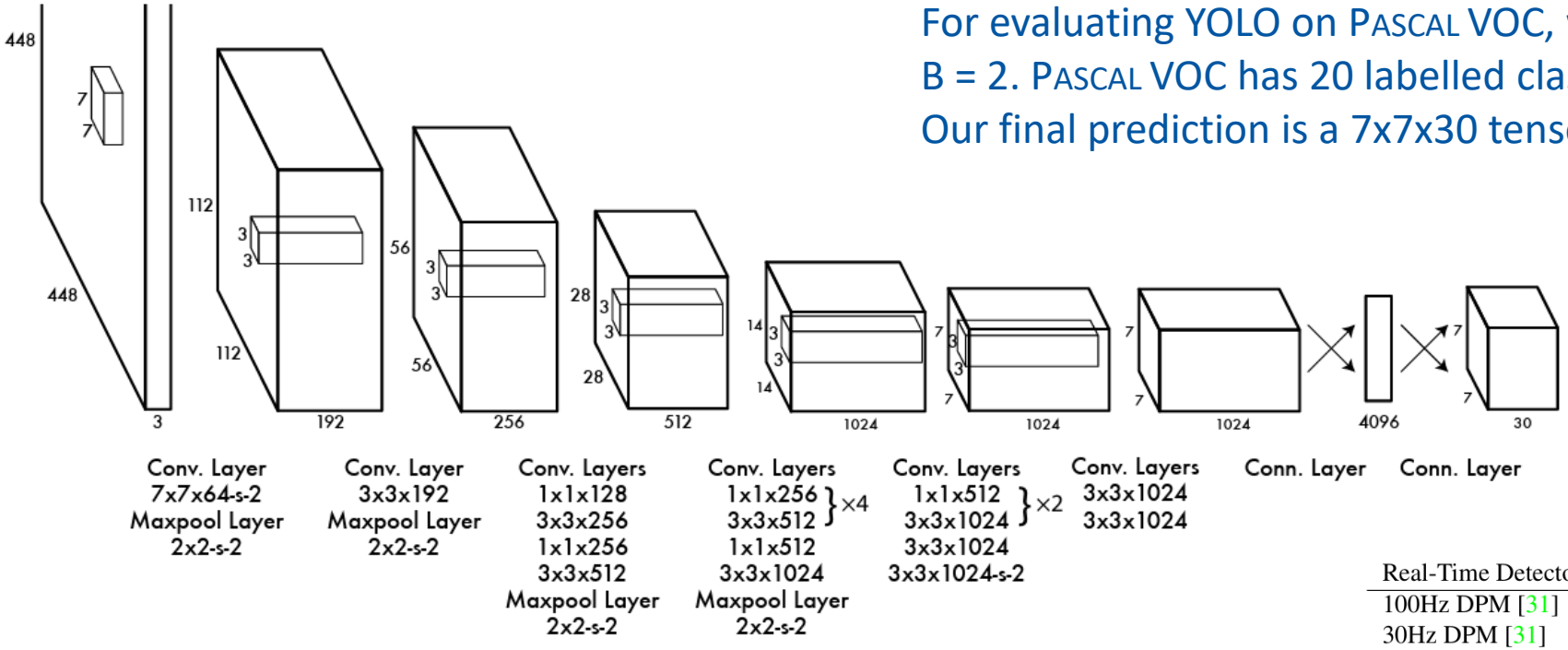
YOLO history

Yolo v1 (darknet) - Joseph Redmon
Yolo v2 (darknet)- Joseph Redmon
Yolo v3 (darknet)- Joseph Redmon
Yolo v3 (pytorch) -Glenn Jocher (ultralytics company founder)
yolo v4 - Alexey Bochkovskiy
yolo v5 - Glenn Jocher
yolo v6 - Meituan technical team
yolo v7 - Alexey Bochkovskiy
yolo v8,v9,v10,v11 - Ultralytics

Source: https://www.reddit.com/r/computervision/comments/1gxce90/yolo_is_not_actually_opensource_and_you_cant_use/

You Only Look Once: Unified real-time object detection, Redmon et al., 2015

YOLOv1 architecture



These predictions are encoded as an Sx Sx (Bx5 + C) tensor.
For evaluating YOLO on PASCAL VOC, we use S = 7,
B = 2. PASCAL VOC has 20 labelled classes so C = 20.
Our final prediction is a 7x7x30 tensor.

Real-Time Detectors	Train	mAP	FPS
100Hz DPM [31]	2007	16.0	100
30Hz DPM [31]	2007	26.1	30
Fast YOLO	2007+2012	52.7	**155**
YOLO	2007+2012	**63.4**	45

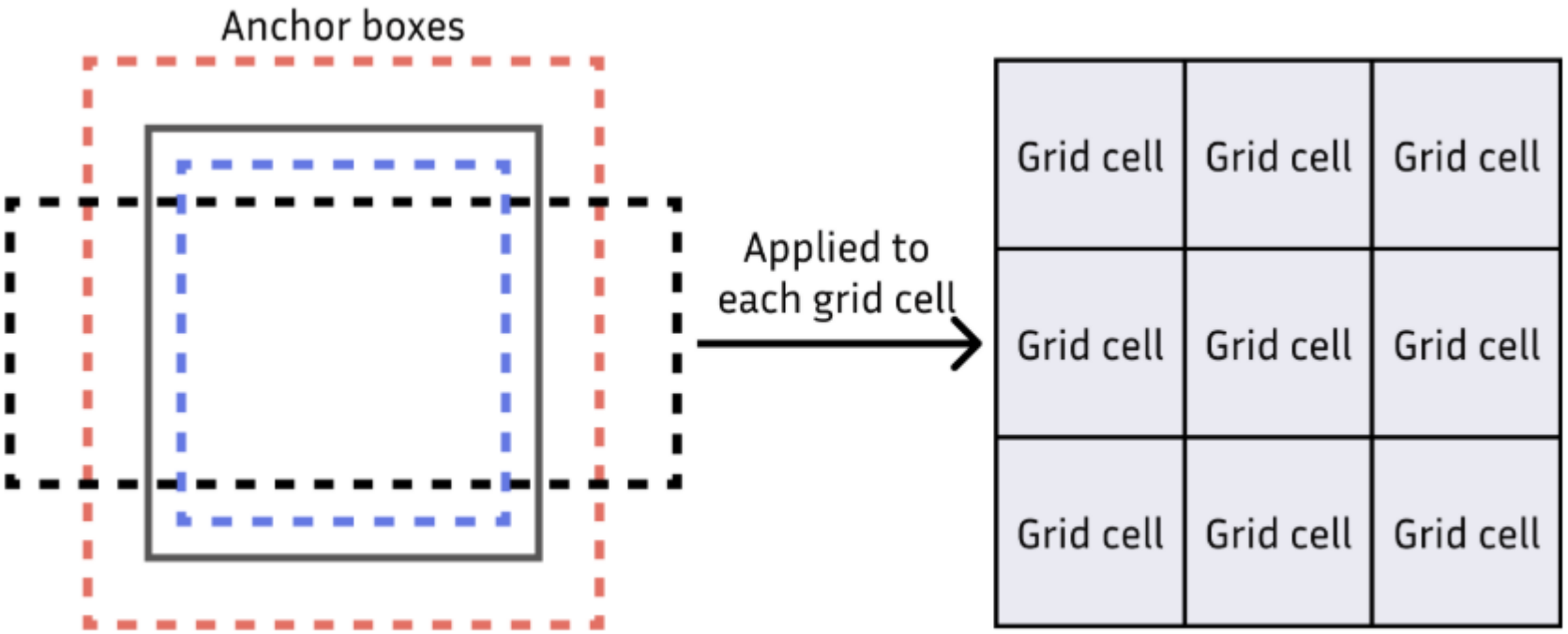
YOLO9000: Better, Faster, Stronger, Redmon et al., 2016

Darnet19 architecture in Yolov2

Type	Filters	Size/Stride	Output
Convolutional	32	3×3	224×224
Maxpool		$2 \times 2/2$	112×112
Convolutional	64	3×3	112×112
Maxpool		$2 \times 2/2$	56×56
Convolutional	128	3×3	56×56
Convolutional	64	1×1	56×56
Convolutional	128	3×3	56×56
Maxpool		$2 \times 2/2$	28×28
Convolutional	256	3×3	28×28
Convolutional	128	1×1	28×28
Convolutional	256	3×3	28×28
Maxpool		$2 \times 2/2$	14×14
Convolutional	512	3×3	14×14
Convolutional	256	1×1	14×14
Convolutional	512	3×3	14×14
Convolutional	256	1×1	14×14
Convolutional	512	3×3	14×14
Maxpool		$2 \times 2/2$	7×7
Convolutional	1024	3×3	7×7
Convolutional	512	1×1	7×7
Convolutional	1024	3×3	7×7
Convolutional	512	1×1	7×7
Convolutional	1024	3×3	7×7
Convolutional	1000	1×1	7×7
Avgpool		Global	1000
Softmax			

YOLO9000: Better, Faster, Stronger, Redmon et al., 2016

Anchor boxes



(a) Assignment of anchor boxes to each grid cell.

Bounding box prediction

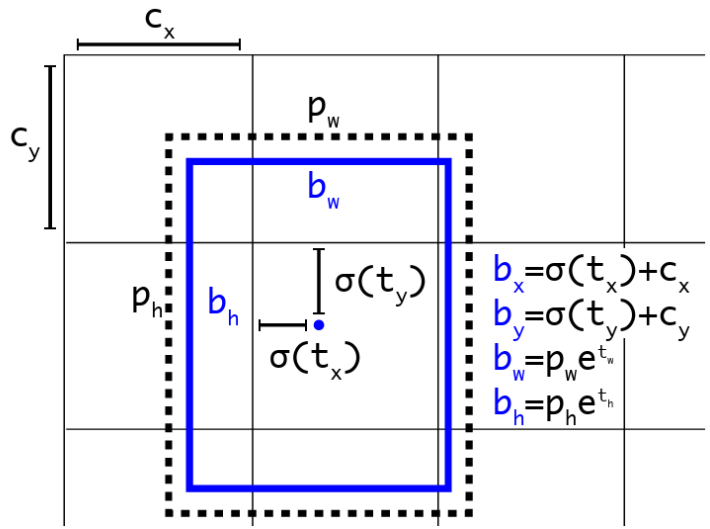


Figure 3: Bounding boxes with dimension priors and location prediction. We predict the width and height of the box as offsets

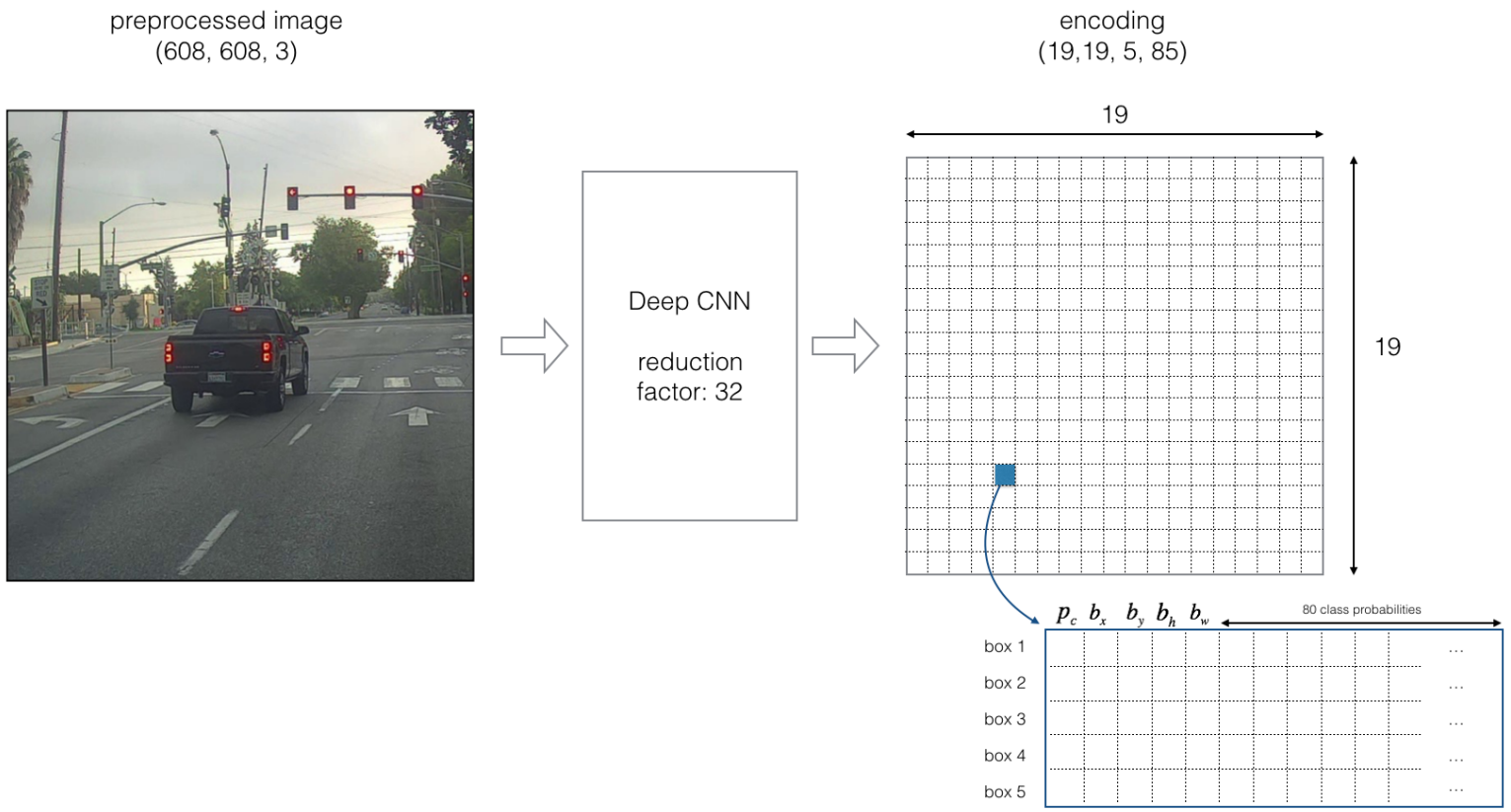
$$b_x = \sigma(t_x) + c_x$$
$$b_y = \sigma(t_y) + c_y$$
$$b_w = p_w e^{t_w}$$
$$b_h = p_h e^{t_h}$$

Darnet53 architecture in Yolov3

	Type	Filters	Size	Output
	Convolutional	32	3 × 3	256 × 256
	Convolutional	64	3 × 3 / 2	128 × 128
1×	Convolutional	32	1 × 1	
	Convolutional	64	3 × 3	
	Residual			128 × 128
	Convolutional	128	3 × 3 / 2	64 × 64
2×	Convolutional	64	1 × 1	
	Convolutional	128	3 × 3	
	Residual			64 × 64
	Convolutional	256	3 × 3 / 2	32 × 32
8×	Convolutional	128	1 × 1	
	Convolutional	256	3 × 3	
	Residual			32 × 32
	Convolutional	512	3 × 3 / 2	16 × 16
8×	Convolutional	256	1 × 1	
	Convolutional	512	3 × 3	
	Residual			16 × 16
	Convolutional	1024	3 × 3 / 2	8 × 8
4×	Convolutional	512	1 × 1	
	Convolutional	1024	3 × 3	
	Residual			8 × 8
	Avgpool		Global	
	Connected		1000	
	Softmax			

	backbone	AP	AP_{50}	AP_{75}	AP_S	AP_M	AP_L
Two-stage methods							
Faster R-CNN+++ [5]	ResNet-101-C4	34.9	55.7	37.4	15.6	38.7	50.9
Faster R-CNN w FPN [8]	ResNet-101-FPN	36.2	59.1	39.0	18.2	39.0	48.2
Faster R-CNN by G-RMI [6]	Inception-ResNet-v2 [21]	34.7	55.5	36.7	13.5	38.1	52.0
Faster R-CNN w TDM [20]	Inception-ResNet-v2-TDM	36.8	57.7	39.2	16.2	39.8	**52.1**
One-stage methods							
YOLOv2 [15]	DarkNet-19 [15]	21.6	44.0	19.2	5.0	22.4	35.5
SSD513 [11, 3]	ResNet-101-SSD	31.2	50.4	33.3	10.2	34.5	49.8
DSSD513 [3]	ResNet-101-DSSD	33.2	53.3	35.2	13.0	35.4	51.1
RetinaNet [9]	ResNet-101-FPN	39.1	59.1	42.3	21.8	42.7	50.2
RetinaNet [9]	ResNeXt-101-FPN	**40.8**	**61.1**	**44.1**	**24.1**	**44.2**	51.2
YOLOv3 608 × 608	Darknet-53	33.0	57.9	34.4	18.3	35.4	41.9

Encoding architecture for YOLOv3



Source: Deep Learning, Andrew Ng

Datasets for Object detection

```
# Classes
0: aeroplane
1: bicycle
2: bird
3: boat
4: bottle
5: bus
6: car
7: cat
8: chair
9: cow
10: diningtable
11: dog
12: horse
13: motorbike
14: person
15: pottedplant
16: sheep
17: sofa
18: train
19: tvmonitor
```

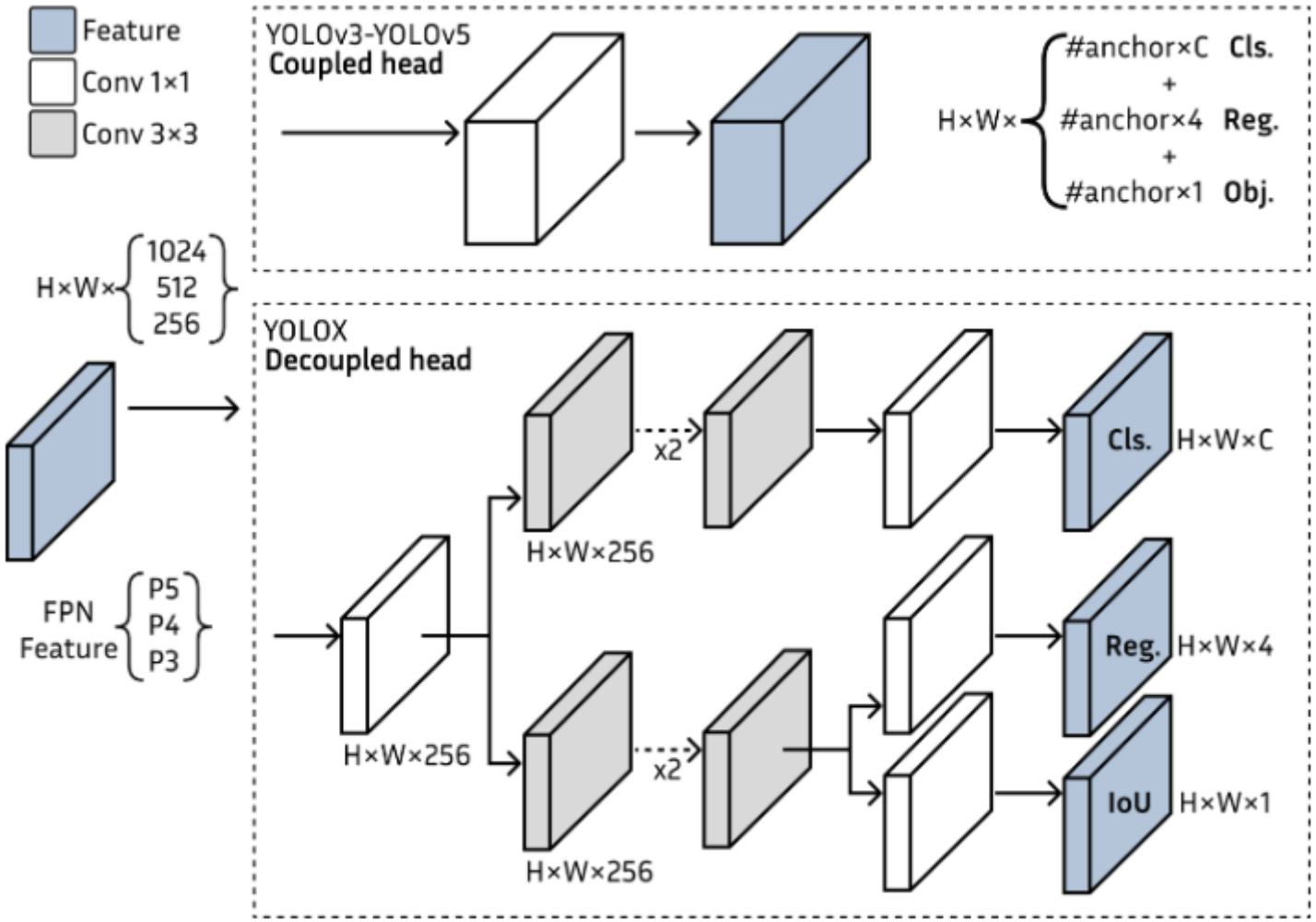
PASCAL VOC (Visual Object Classes)

- 20 classes
- Voc2012: 5,717 (train), 5,823 (val), 11,540 (trainval), 5,085 (test)
- Voc2007: 2,501 (train), 2,510 (val), 5,011 (trainval), 4,952 (test)

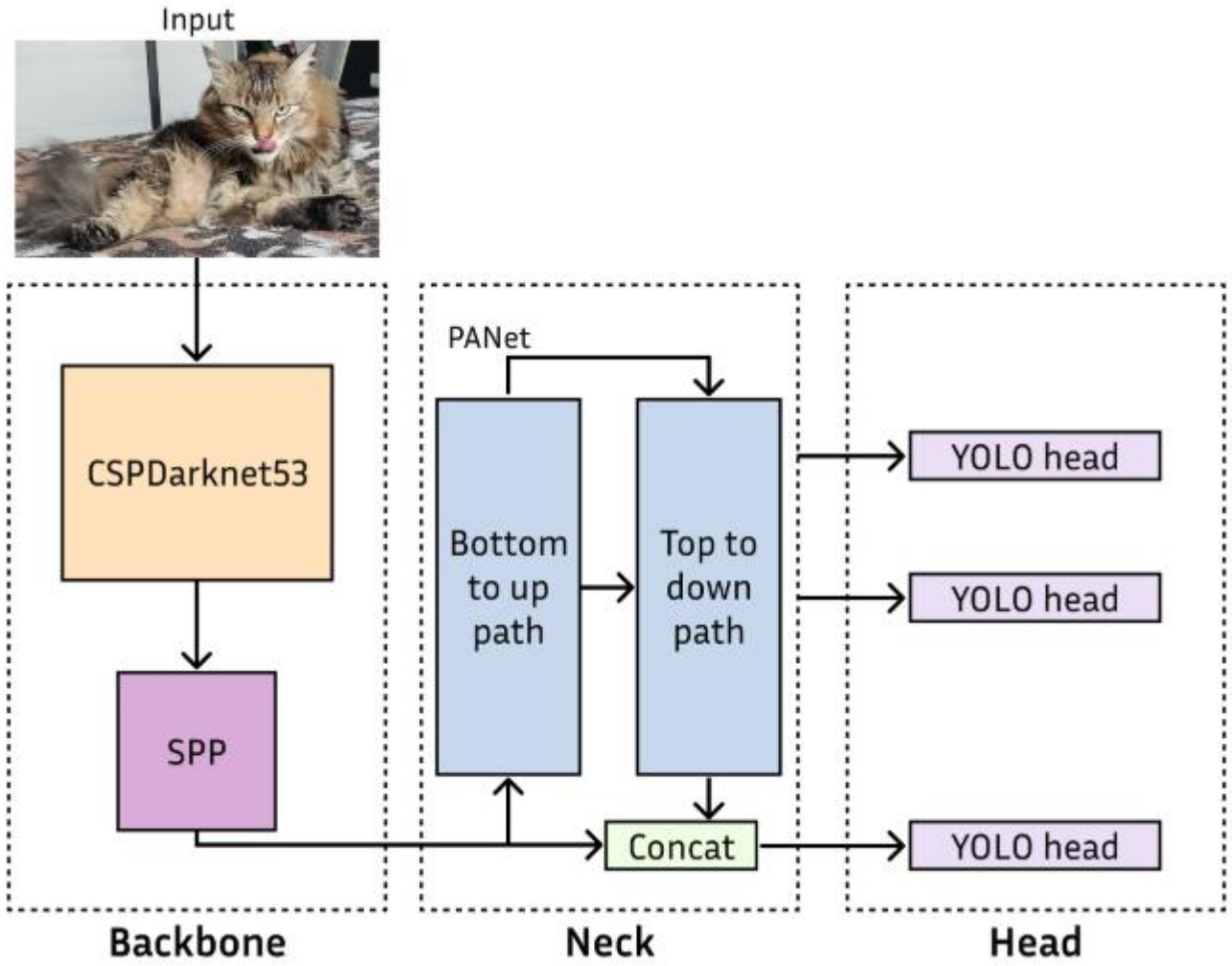
COCO

- 80 classes
- Coco2017: 118,287(train), 5,000 (validation), 40,670 (test)
- ~~Coco2014: 82,783 (train), 40500 (validation), 40775 (test)~~

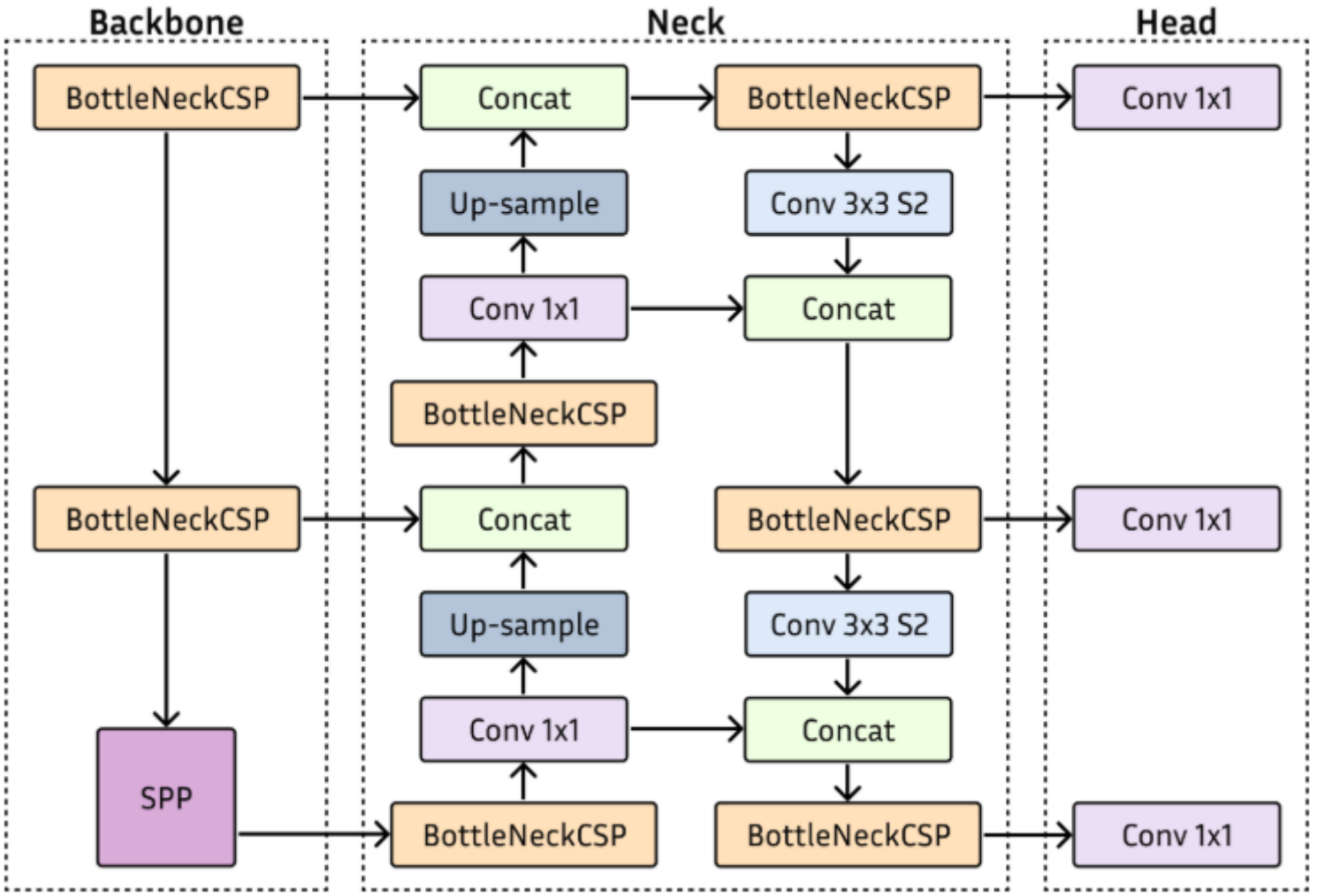
head


Feature
Conv 1×1
Conv 3×3
YOLOv3-YOLOv5
Coupled head
H×W×
#anchor×C Cls.
+
#anchor×4 Reg.
+
#anchor×1 Obj.
H×W×
1024
512
256
YOLOX
Decoupled head
x2
Cls.
H×W×C
H×W×256
FPN
Feature
P5
P4
P3
H×W×256
x2
Reg.
H×W×4
IoU
H×W×1
H×W×256

Yolov4

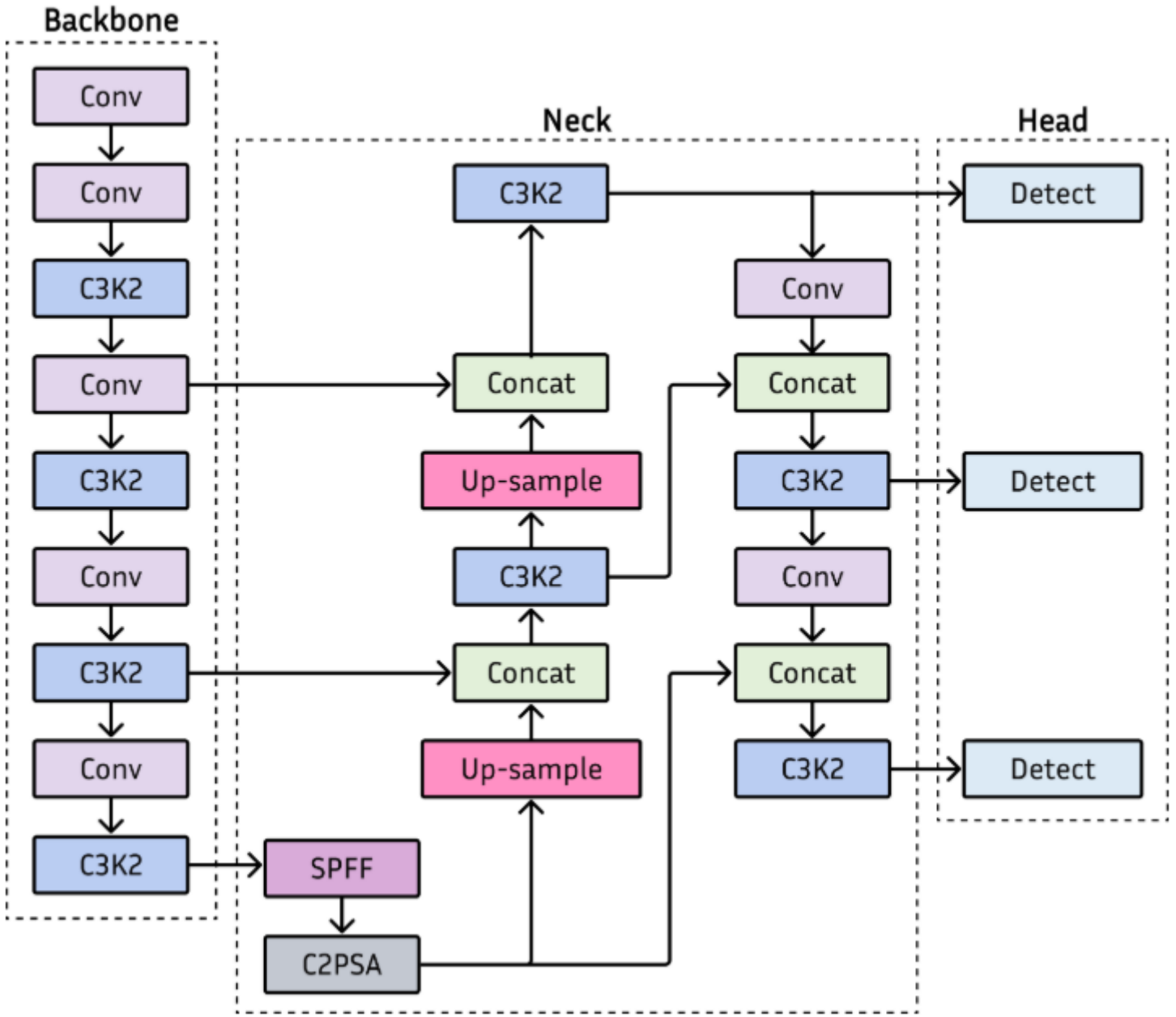

Input
CSPDarknet53
SPP
PANet
Bottom to up path
Top to down path
Concat
YOLO head
YOLO head
YOLO head
Backbone
Neck
Head

Yolov5


Backbone
Neck
Head
BottleNeckCSP
BottleNeckCSP
SPP
Concat
Up-sample
Conv 1x1
BottleNeckCSP
Concat
Up-sample
Conv 1x1
BottleNeckCSP
BottleNeckCSP
Conv 3x3 S2
Concat
BottleNeckCSP
Conv 3x3 S2
Concat
BottleNeckCSP
Conv 1x1
Conv 1x1
Conv 1x1

A Decade of You Only Look Once (YOLO) for Object Detection, arxiv.2504.18586

Yolov11



Approach to Object Detection

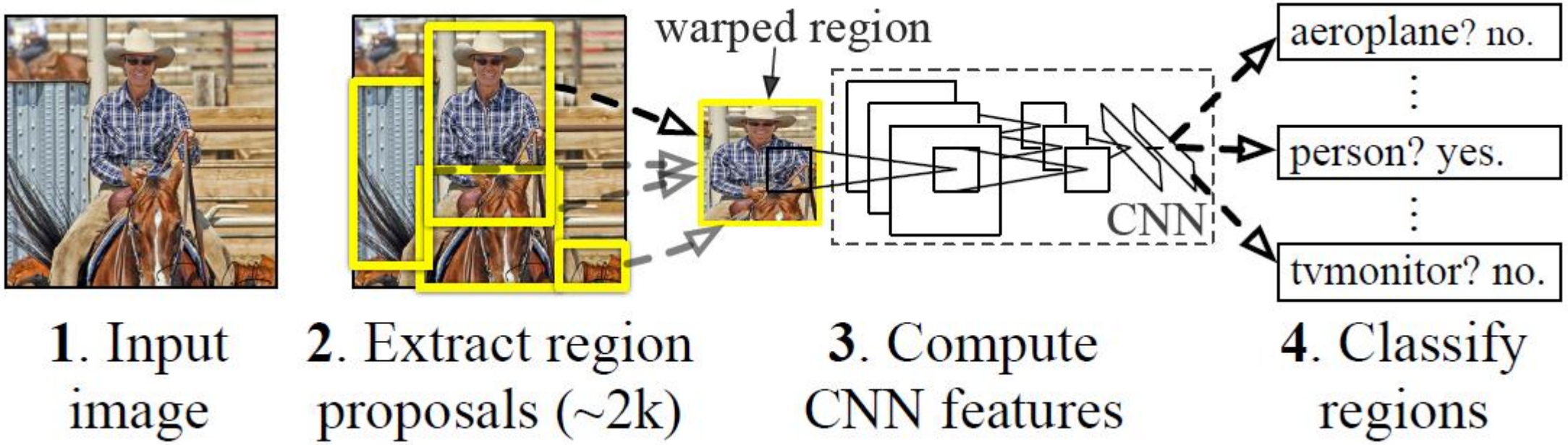
- Object detection involves localizing objects and identifying them in images.
- Two primary approaches:
  - **One-stage methods** (faster, slightly less accurate). :
    - SSD, RetinaNet, YOLO
  - **Two-stage methods** (more accurate, slower):
    - R-CNN, Fast R-CNN, Faster R-CNN

Yolo

- Grid: SxS
- Anchor: 2 ~ 5 (predefined bounding boxes)
- backbone/neck/head: Predict bounding boxes and classes

Rich feature hierarchies for accurate object detection and semantic segmentation, 1311.2524, 2014

R-CNN: Regions with CNN features



R-CNN

- Region Proposal Network: Predict 2000 Region of Interest(ROI)
- Backbone: feature extraction/classification/localization

Rich feature hierarchies for accurate object detection and semantic segmentation, 1311.2524, 2014

R-CNN: Regions with CNN features

1. Selective search to identify regions of interest

   This step uses a computer vision-based algorithm capable of extracting candidate regions.

   Selective search generates around 2,000 region proposals per image.

2. Feature extraction

   A deep convolution neural network extracts features from each region of interest.

   Since deep neural networks typically take in fixed-sized inputs, the regions are warped into a fixed size before being fed into the deep neural network.

3. Classification/Localization

   A class-specific support vector machine (SVM) is trained on the extracted features to classify the region. Additionally, bounding-box regressors are added to fine-tune the object's location within the region.

Source: Math and Architectures of Deep Learning, Krishenedu Chauhury

Fast R-CNN

Problems: One of the biggest disadvantages of the R-CNN-based approach is to extract features for every region proposal independently.

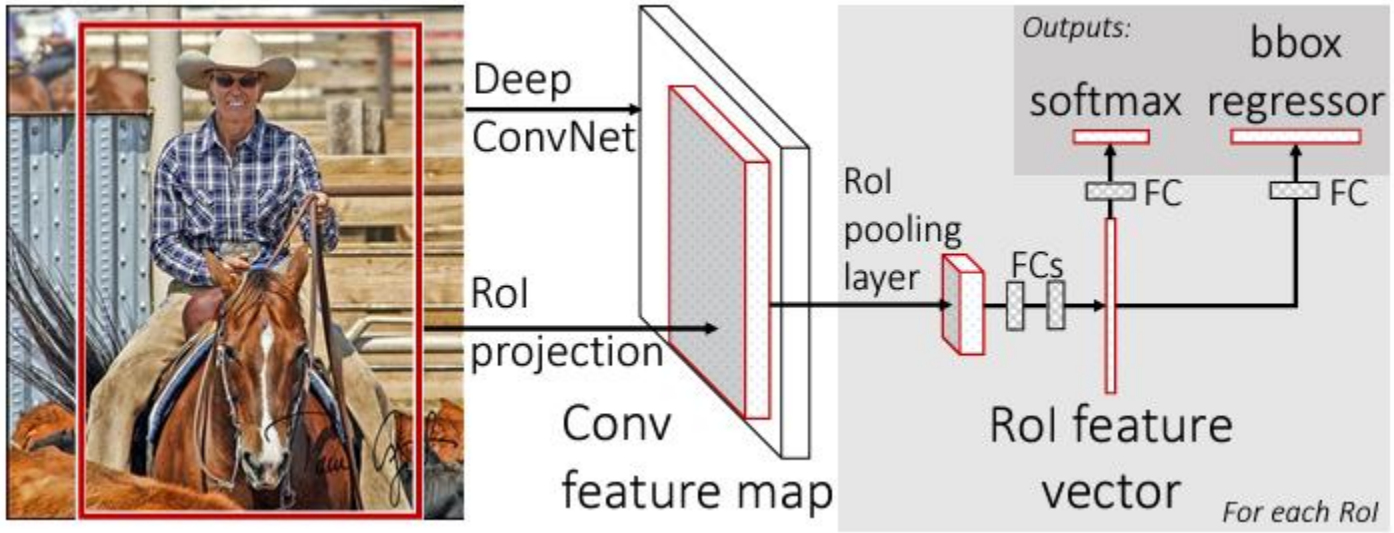
- 2000 region proposal → 2000 forward passes to extract feature
- Region of interest (RoI) pooling

  The Fast R-CNN uses the entire image as the input to the CNN instead of a single region proposal.
- Multitask loss

  The Fast R-CNN eliminates the need to use SVMs. Instead, the deep neural network does both classification and bounding-box regression.

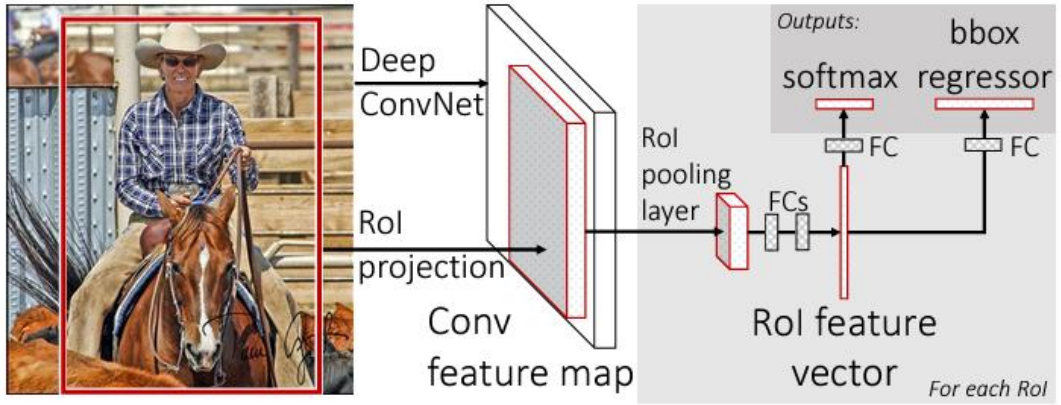
Source: Math and Architectures of Deep Learning, Krishenedu Chauhury

Fast R-CNN



Source: Fast R-CNN, Ross Girshick

Fast R-CNN



1. An input image and multiple regions of interest (RoIs) are input into a fully convolutional network.
2. Each RoI is pooled into a fixed-size feature map.
3. The RoI Pooling is mapped to a feature vector by fully connected layers (FCs).
4. The network has two output vectors per RoI:
    1. Softmax probabilities
    2. per-class bounding-box regression offsets

Source: Fast R-CNN, Ross Girshick

Faster R-CNN

- Problems: selective search to be run to obtain region proposals on CPU in Fast R-CNN:
- Region proposal network (RPN)

  This is the module responsible for generating the region proposals.

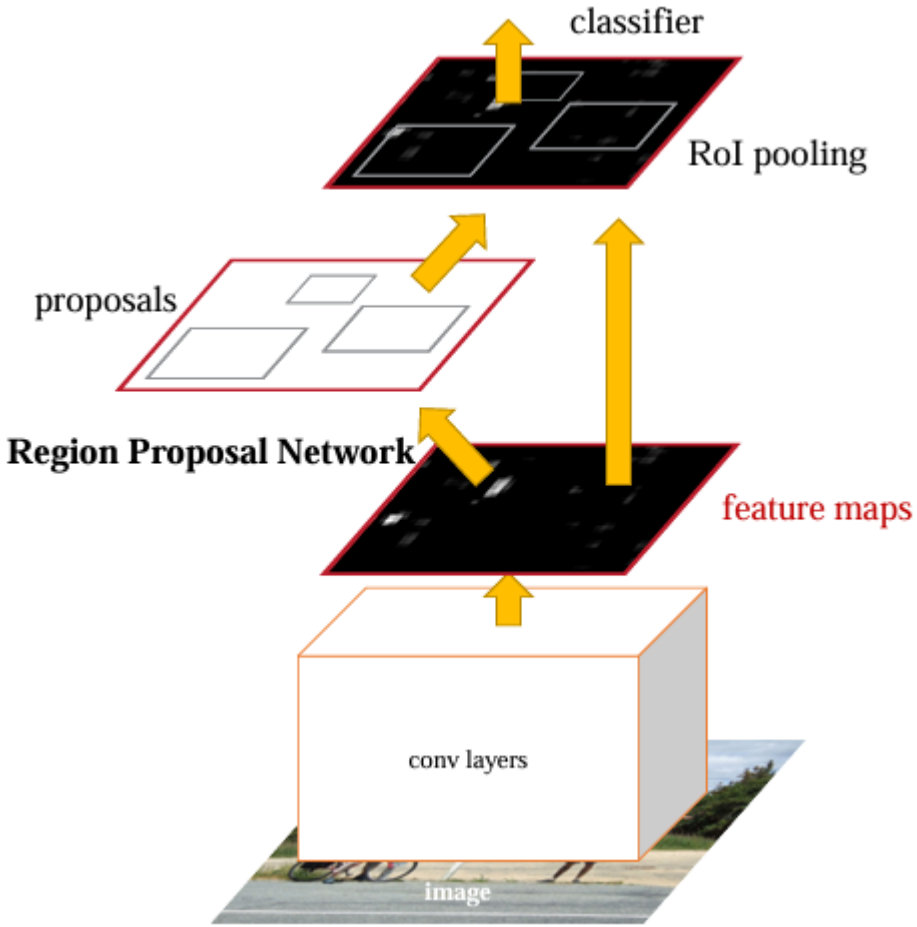
  RPNs are designed to efficiently predict region proposals with a wide range of scales and aspect ratios.
- R-CNN module

  This is the same as the Fast R-CNN.

  It receives a bunch of region proposals and performs RoI pooling followed by classification and regression.
- The RPN and the R-CNN module share the same convolutional layers:

  the weights are shared rather than learning two separate networks.

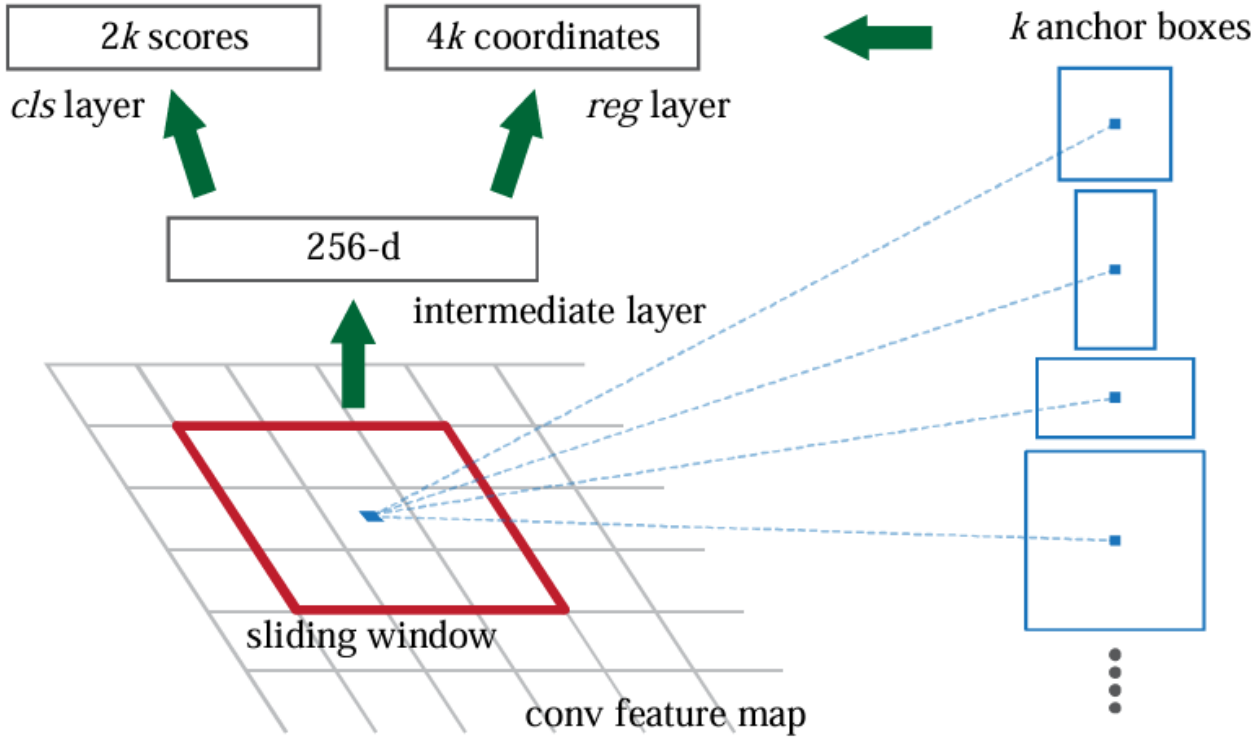
Faster R-CNN = RPN + Fast R-CNN



- Faster R-CNN is a single, unified network for object detection.
- The RPN module serves as the ‘attention’ of this unified network.

Source: Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks

Region Proposal Network (RPN)



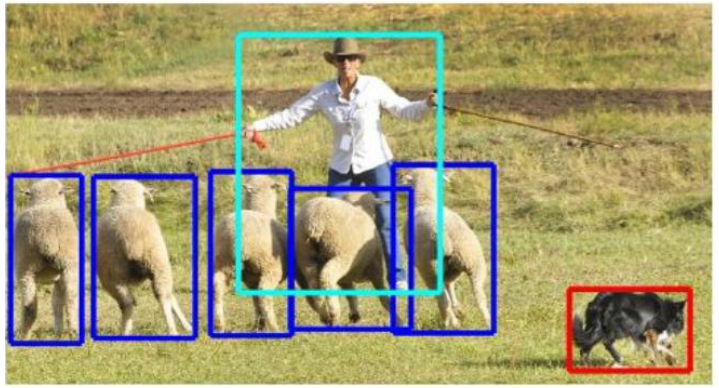
Source: Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks

Topics(2)

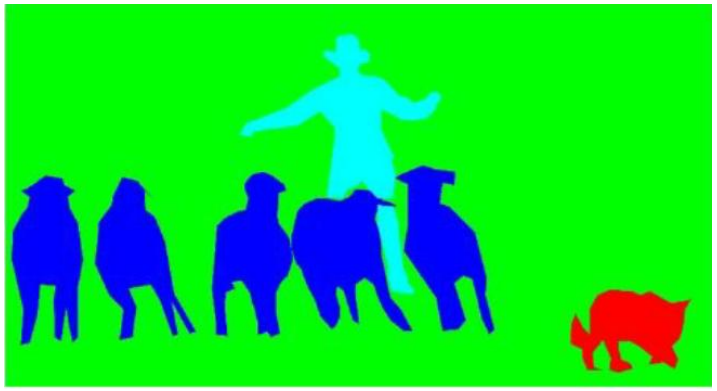
- Semantic Segmentation:
  - Encoder-decoder network
  - Upsampling
  - Transposed convolution

Segmentation

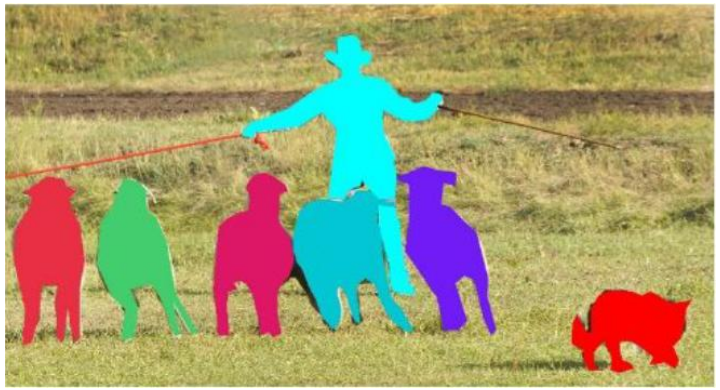
Image classification



object detection



semantic segmentation



instance segmentation

Semantic segmentation in biomedical images

Semantic image segmentation classifies each pixel in an image.

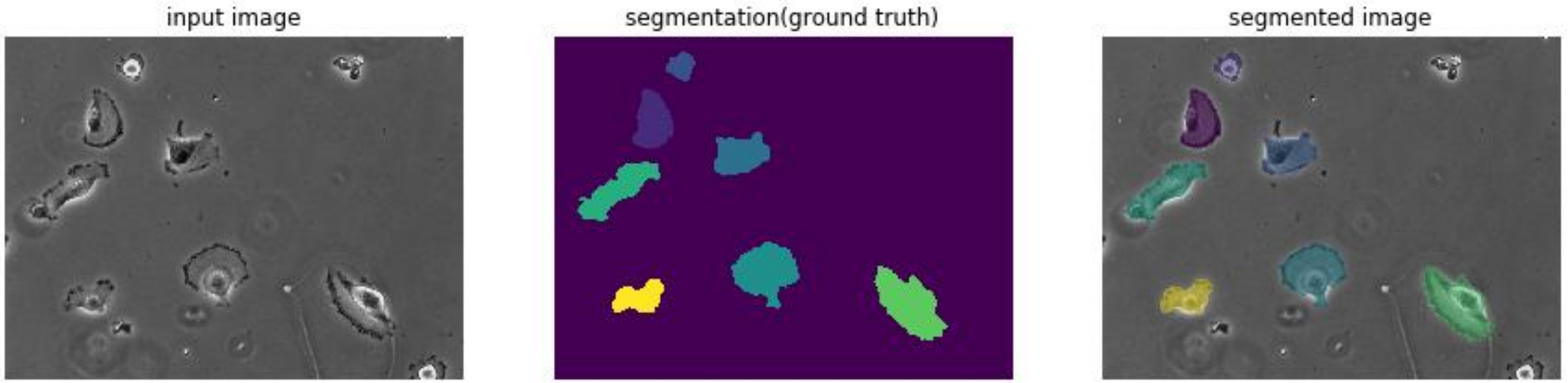


Fig. input image, its segmentation(ground truth), and its segmented image of PhC-U373 dataset

Source: ‘Neural Network Pruning for Biomedical Image Segmentation’, SPIE Medical Imaging 2021

Segmentation using Artificial Neural Networks

Instead of manual segmentation, segmentation using Artificial Neural Networks is very useful in biomedical image analysis.

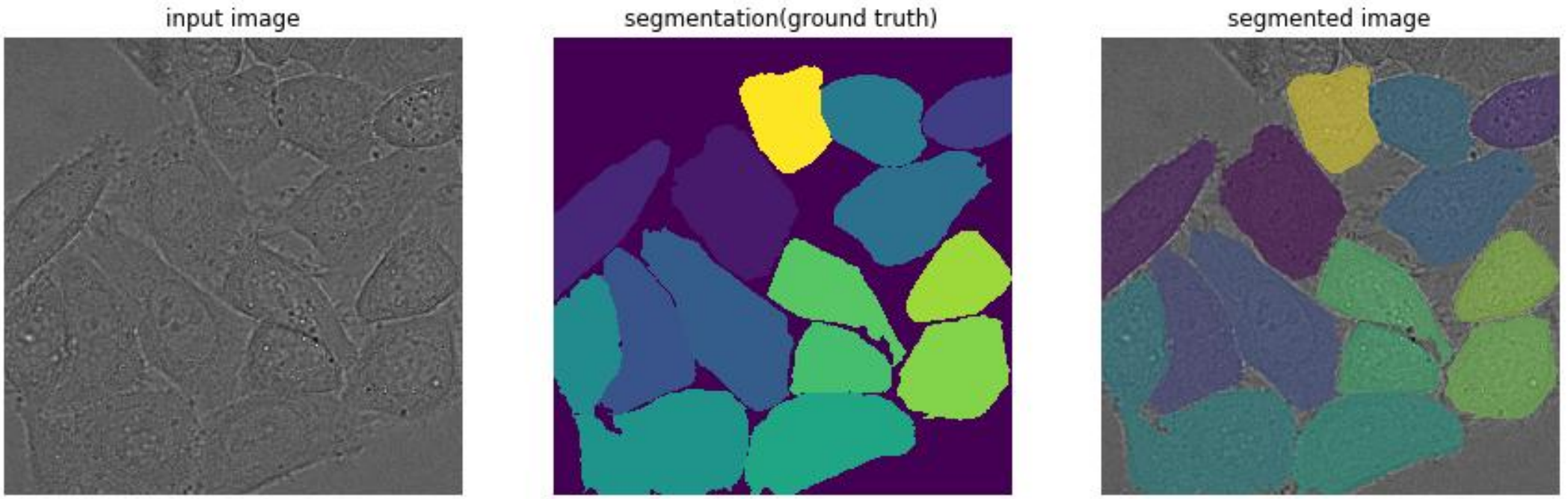
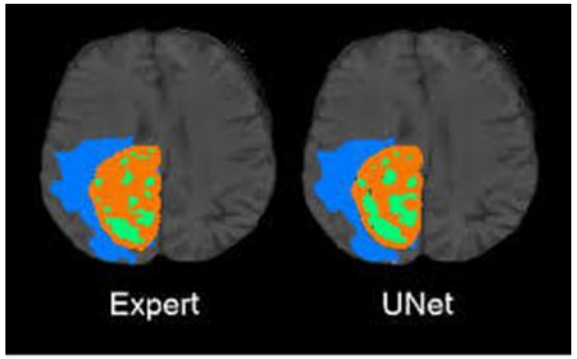


Fig. input image, its segmentation(ground truth), and its segmented image of DIC-HeLa dataset

Source: ‘Neural Network Pruning for Biomedical Image Segmentation’, SPIE Medical Imaging 2021

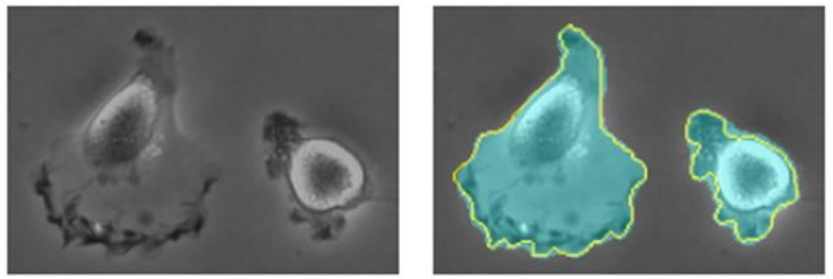
U-Net: Image Segmentation Network



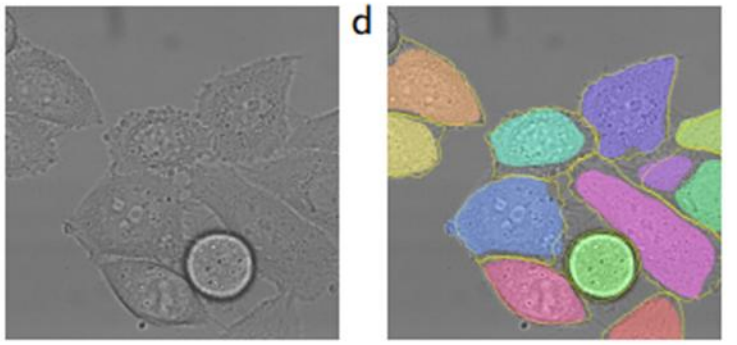
https://neurohive.io/en/popular-networks/u-net/

Segmented CT Scan image

Biomedical cell



Biomedical cell

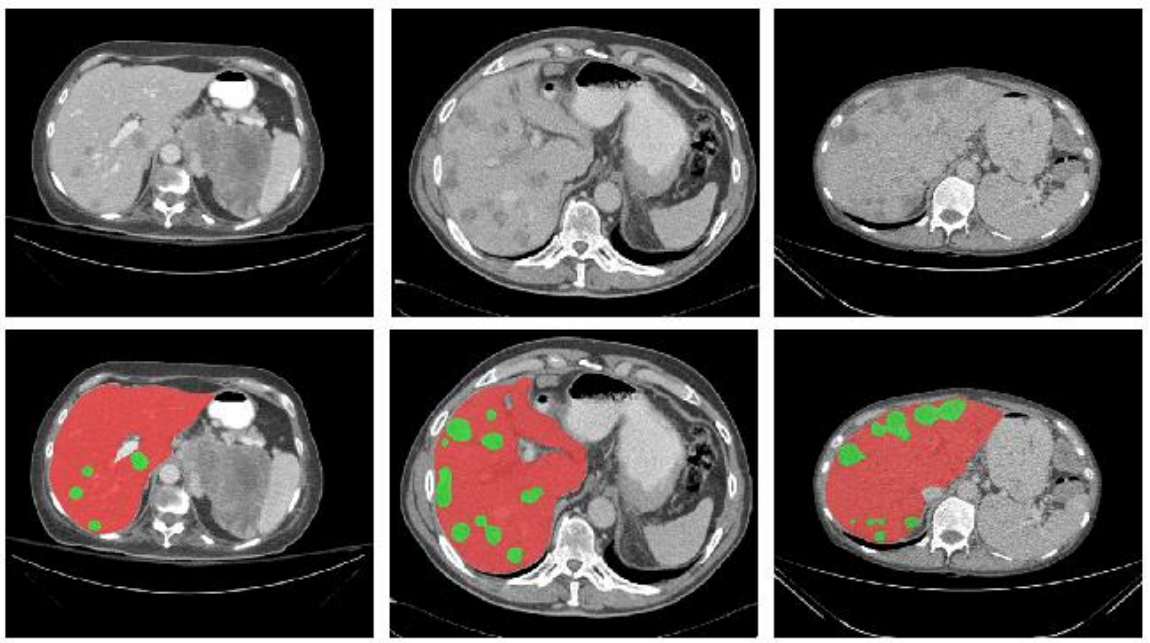


Arxiv: 1611.09326

U-Net: Image Segmentation Network

CT scans images

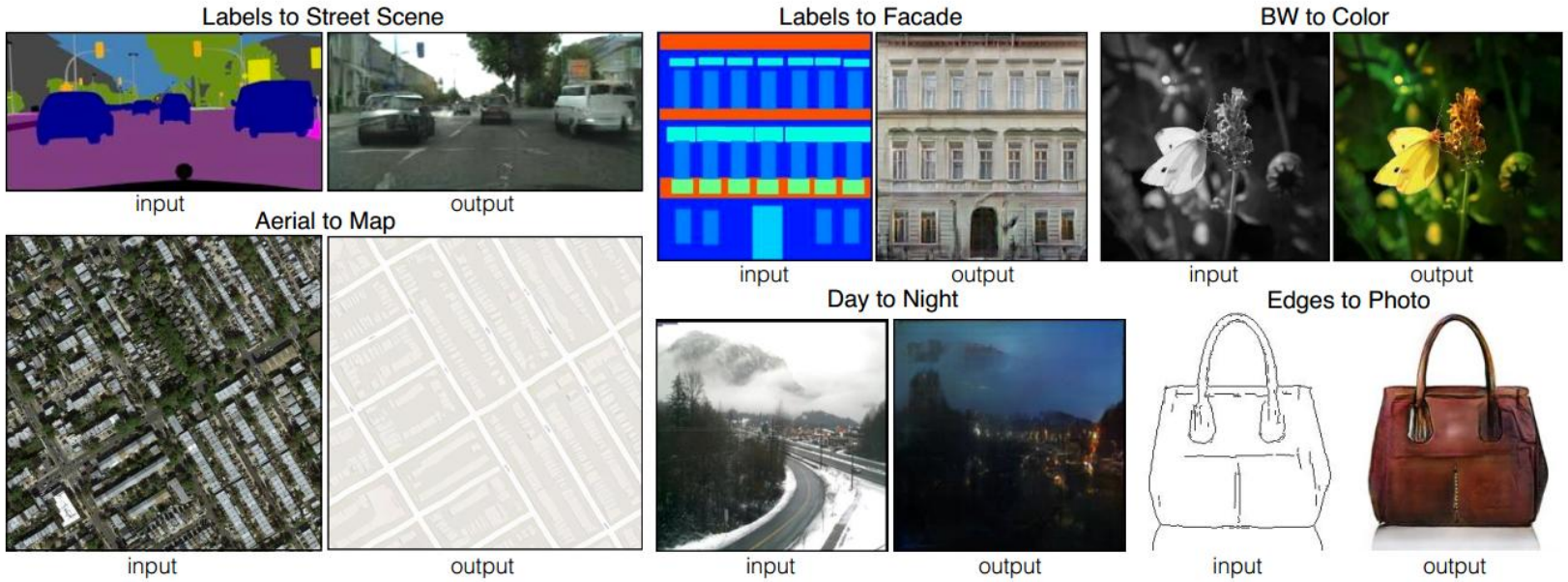
Liver and tumor segmentation results

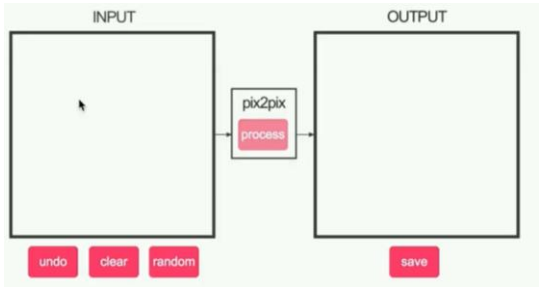


The red regions denote the liver while the green ones denote the lesions.

H-DenseUNet: Hybrid Densely Connected UNet for Liver and Liver Tumor Segmentation from CT Volumes

pix2pix

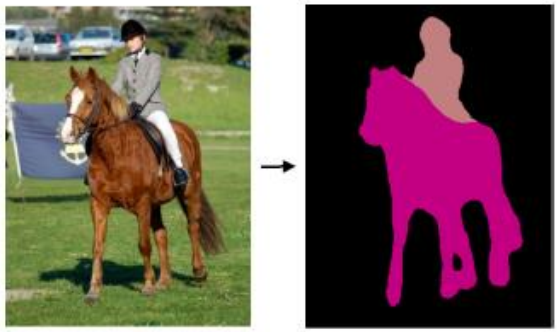




Source: Image-to-image Translation with Conditional Adversarial Nets, CVPR 2017

Image-to-Image Translation

Object labeling



[Long et al. 2015]

Edge Detection

[Xie et al. 2015]

Season change

[Laffont et al. 2014]

Artistic style transfer

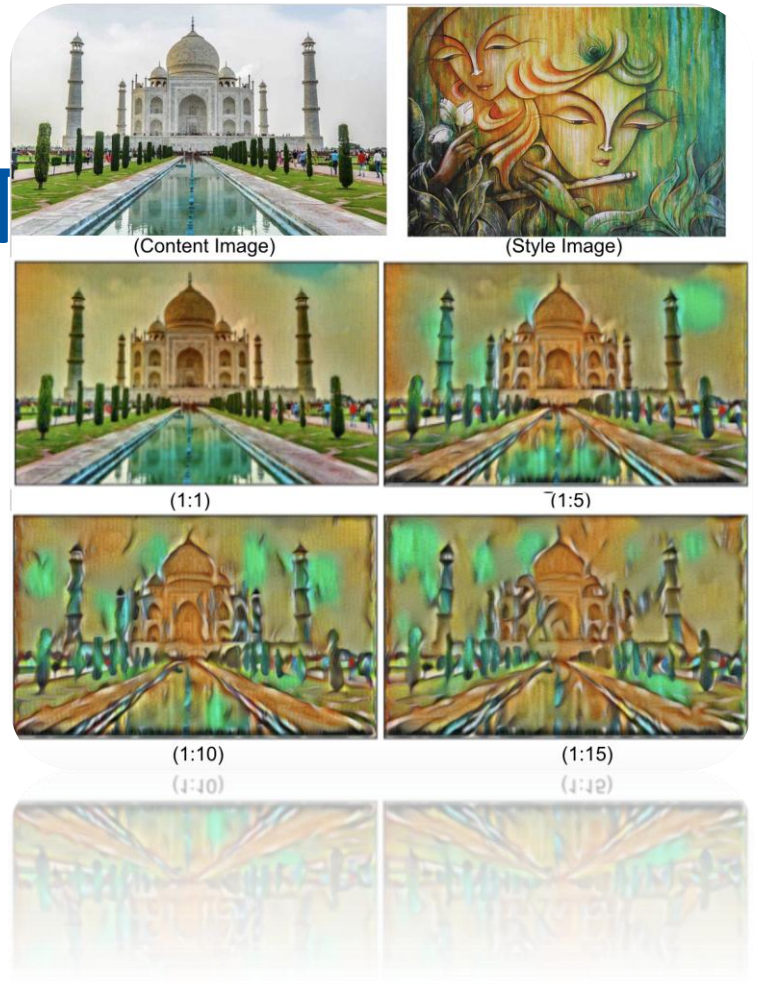
[Gatys et al. 2016]

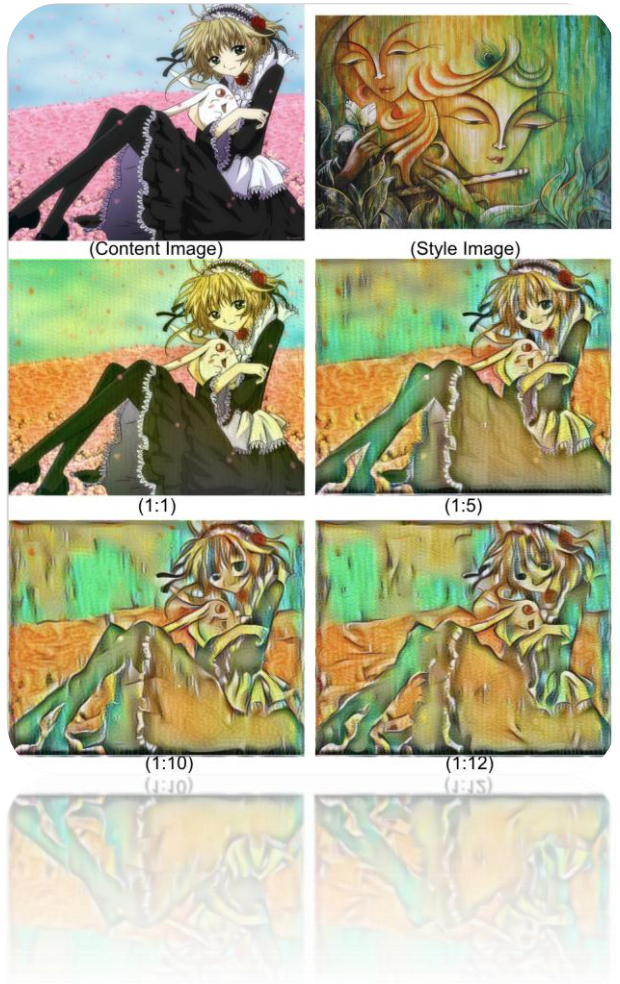
Neural Style Transfer

Content Image Style Image Output Image

Source: ‘Flexible selecting of style to content ratio in Neural Style Transfer’, ICMLA 2018

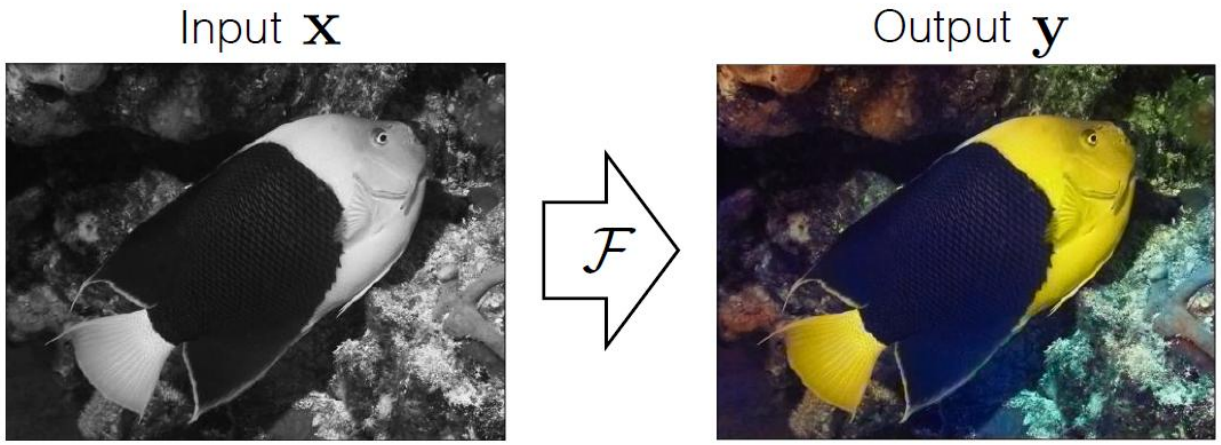
Neural Style T





Source: ‘Flexible selecting of style to content ratio in Neural Style Transfer’, ICMLA 2018

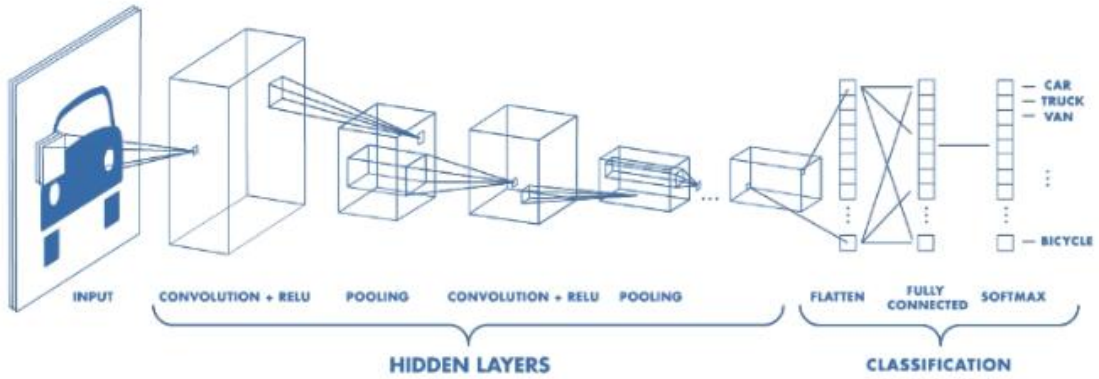
Image colorization



[Zhang et al., ECCV 2016]

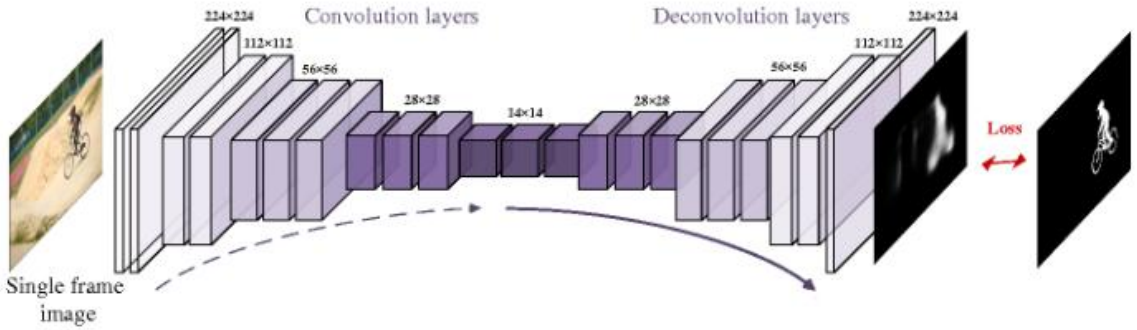
FCN: Fully Convolution Networks

CNN = Encoder + Classifier



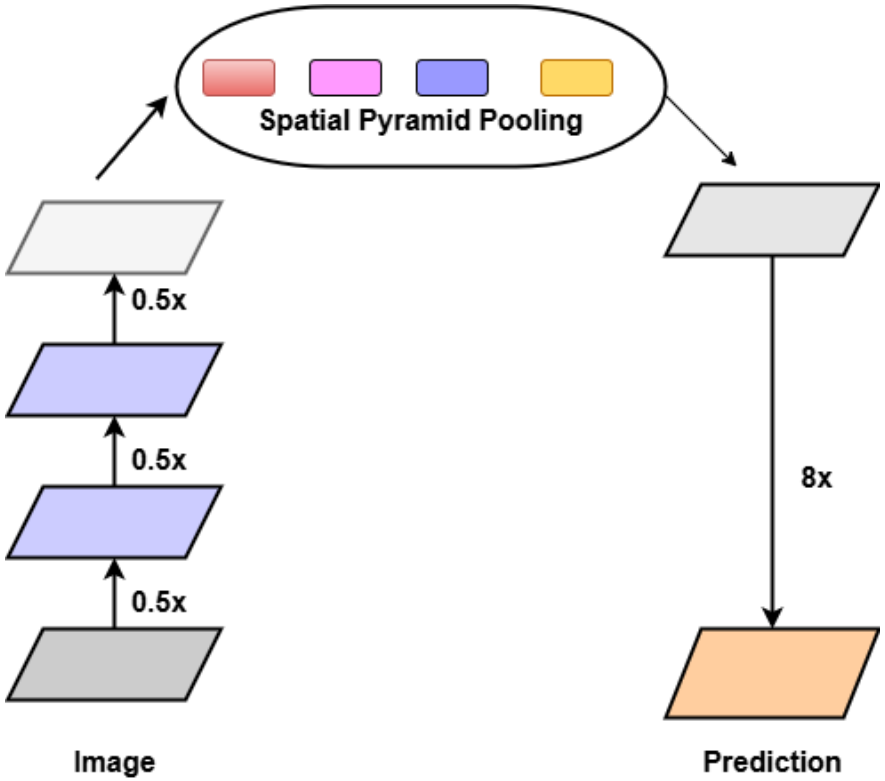
https://www.mathworks.com/solutions/deep-learning/convolutional-neural-network.html

FCN = Encoder + Decoder



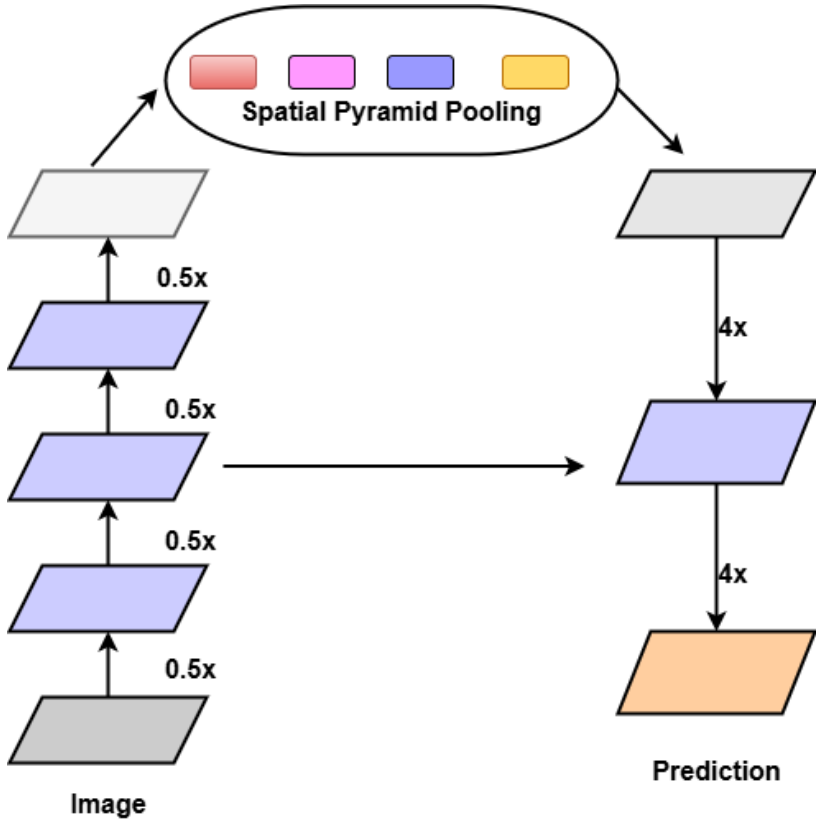
https://www.doc.ic.ac.uk/~jce317/semantic-segmentation.html

Deeplab-v3



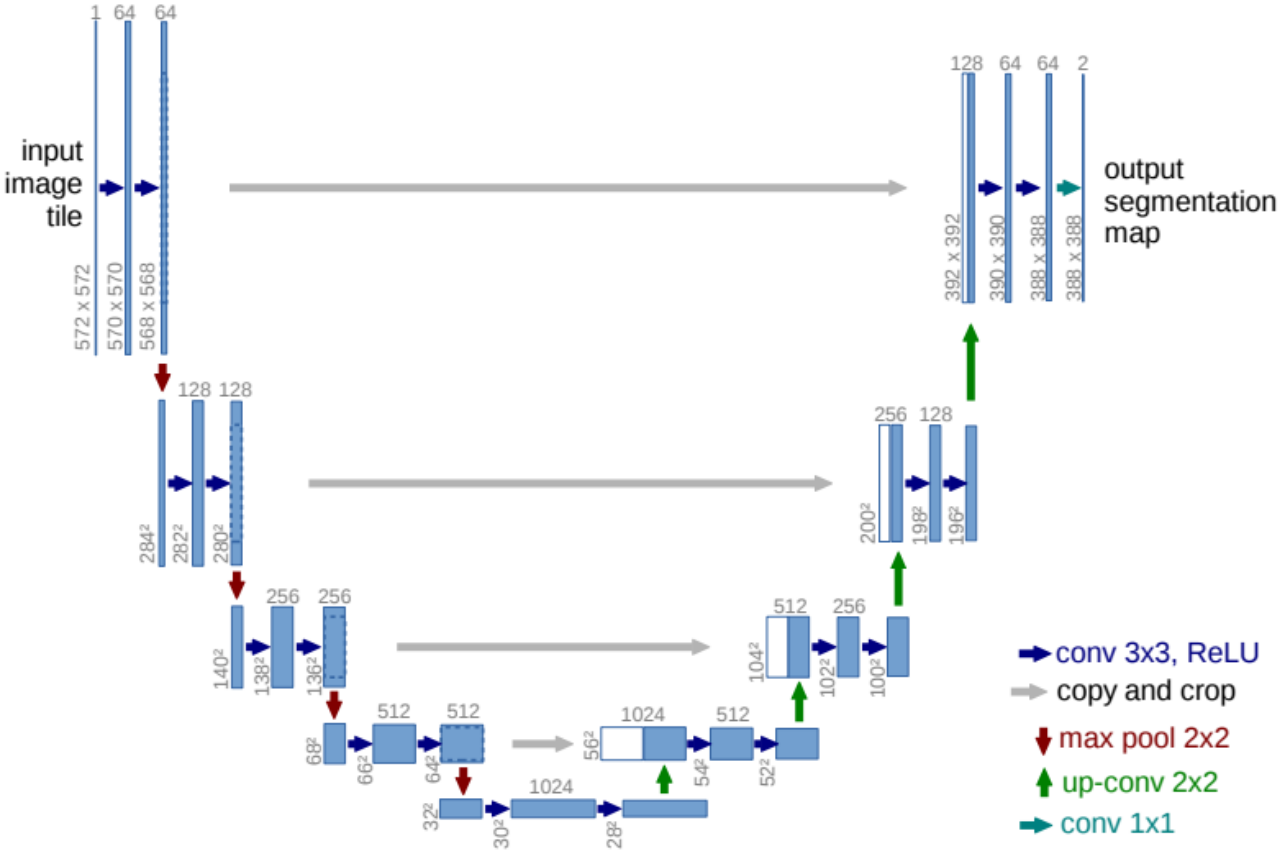
Source: “Rethinking atrous convolution for semantic image segmentation,“ arXiv:1706.05587, 2017

Deeplab-v3+



Source: “Encoderdecoder with atrous separable convolution for semantic image segmentation”, ECCV, 2018

U-Net: Image Segmentation Network

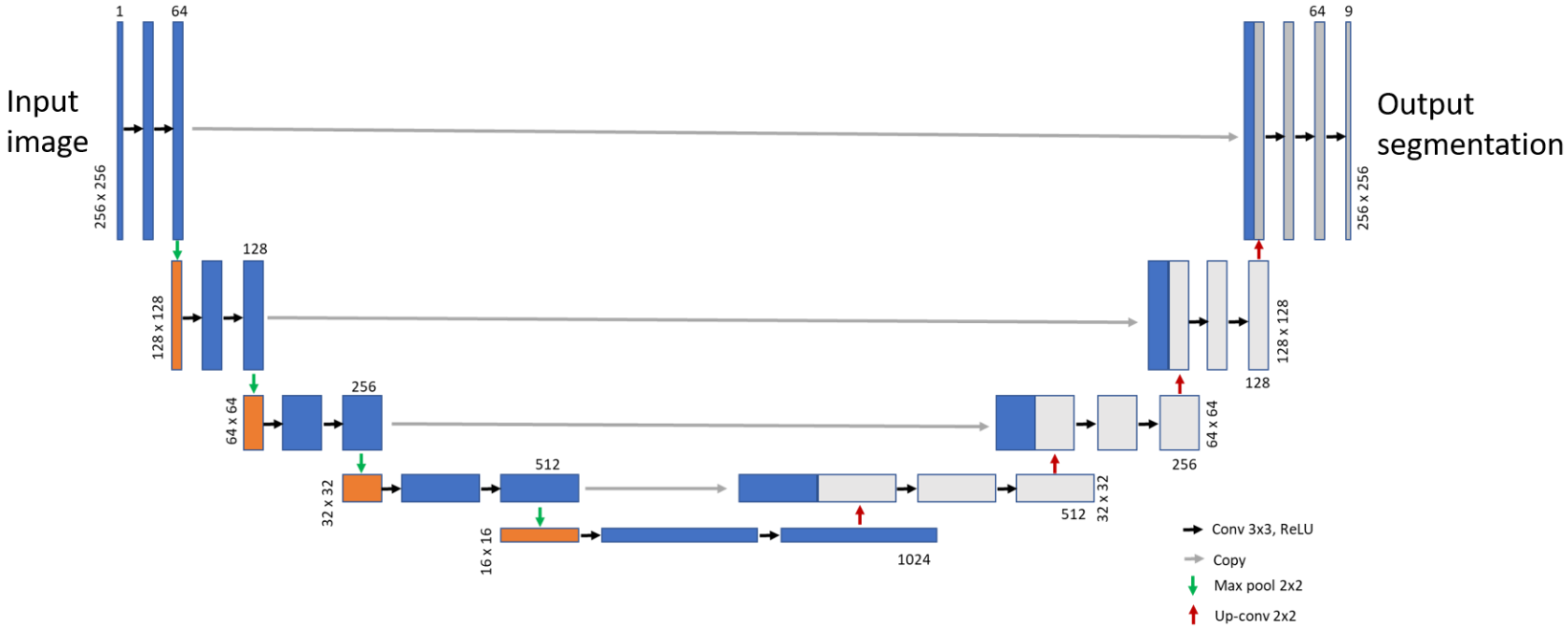


Arxiv: 1611.09326

U-Net

U-Net is a popular semantic segmentation network.

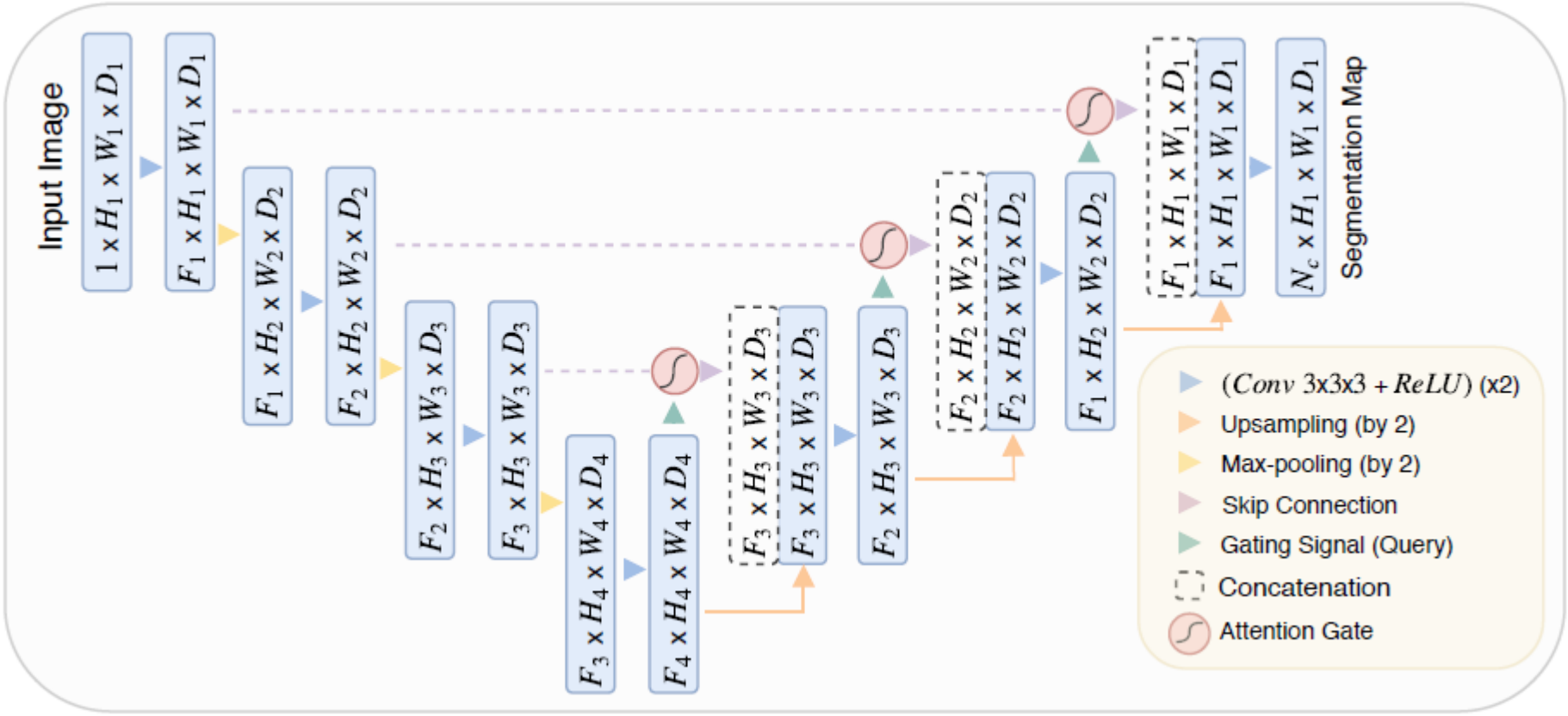
It consists of an encoder-decoder architecture.



Source: 'Neural Network Pruning for Biomedical Image Segmentation', Medical Imaging 2021

SJSU SAN JOSÉ STATE UNIVERSITY

Attention U-Net

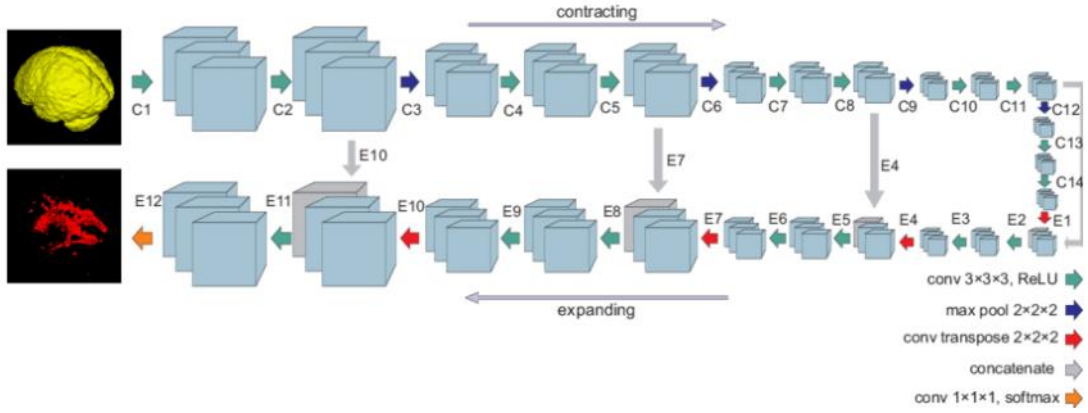


Source: Attention U-Net, arXiv: 1804.03999, 2018

SJSU SAN JOSÉ STATE UNIVERSITY

Down-sample & Up-sample

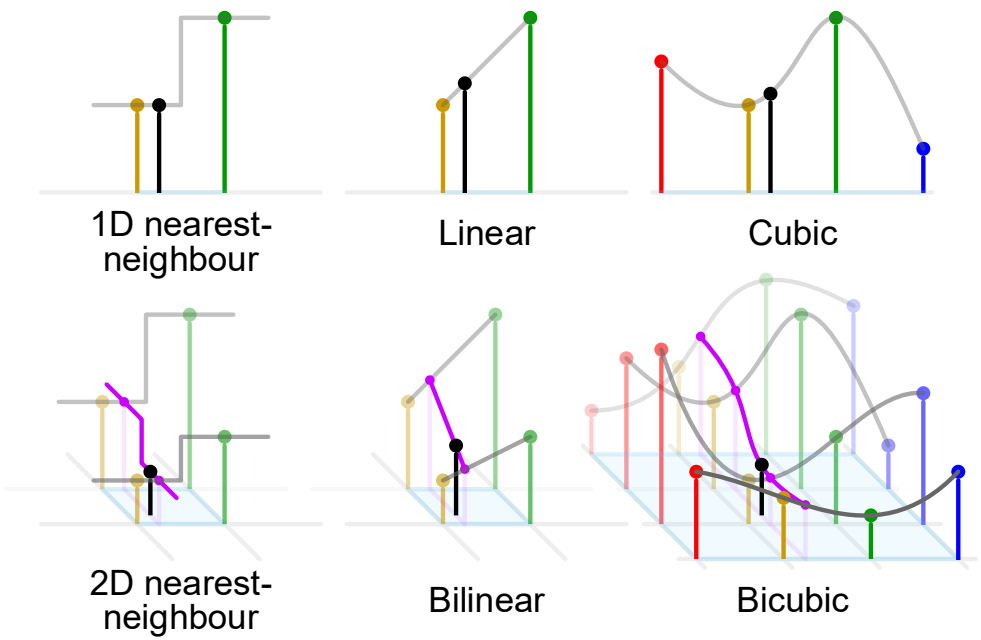
- Encoder
  - Down-sampling
  - 256x256x1→ 256x256x64 → 128x128x128→ 64x64x256 → 32x32x512 → 16x16x1024
- Decoder
  - Up-sampling
  - 256x256x9 ← 256x256x64 ← 128x128x128 ← 64x64x256 ← 32x32x512 ← 16x16x1024

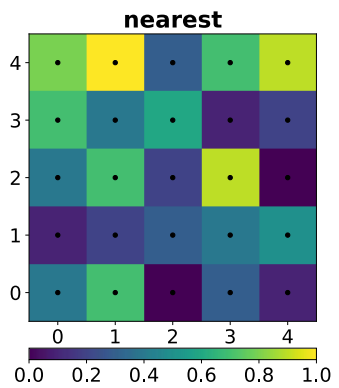


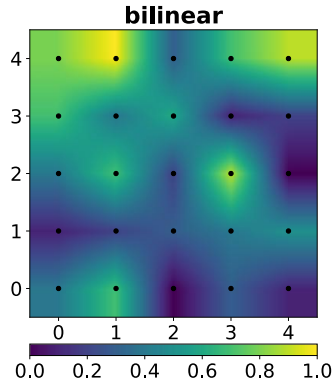
https://neurohive.io/en/popular-networks/u-net/

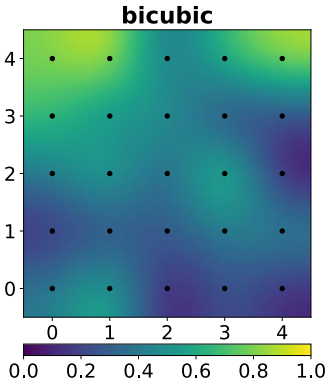
SJSU SAN JOSÉ STATE UNIVERSITY

interpolation techniques









Source: https://en.wikipedia.org/wiki/Bicubic_interpolation

SJSU SAN JOSÉ STATE UNIVERSITY

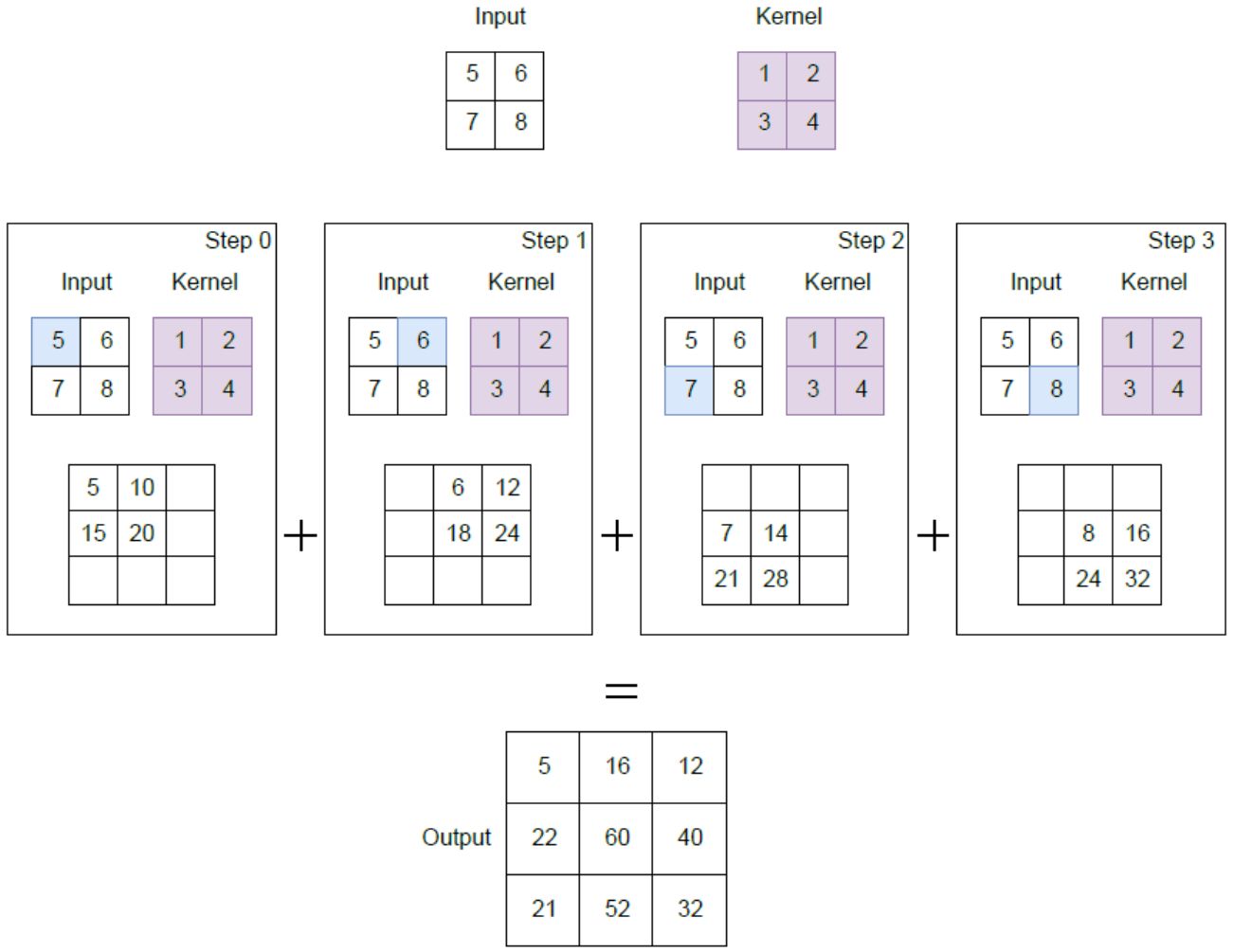
Upsampling (or upscaling)

- The decoder network, which tries to reconstruct the original image from the embedding, has to upscale/upsample a lower-resolution input into a higher-resolution output.
- Many interpolation techniques, such as nearest neighbor, bilinear, and bicubic interpolation, can be used to perform this upsampling operation.
- A more optimal way to perform upsampling is through transpose convolutions, where the mapping function is learned during the training process instead of being predefined.

Source: Math and Architectures of Deep Learning, Krishenedu Chauhury

Transposed convolution: 2D

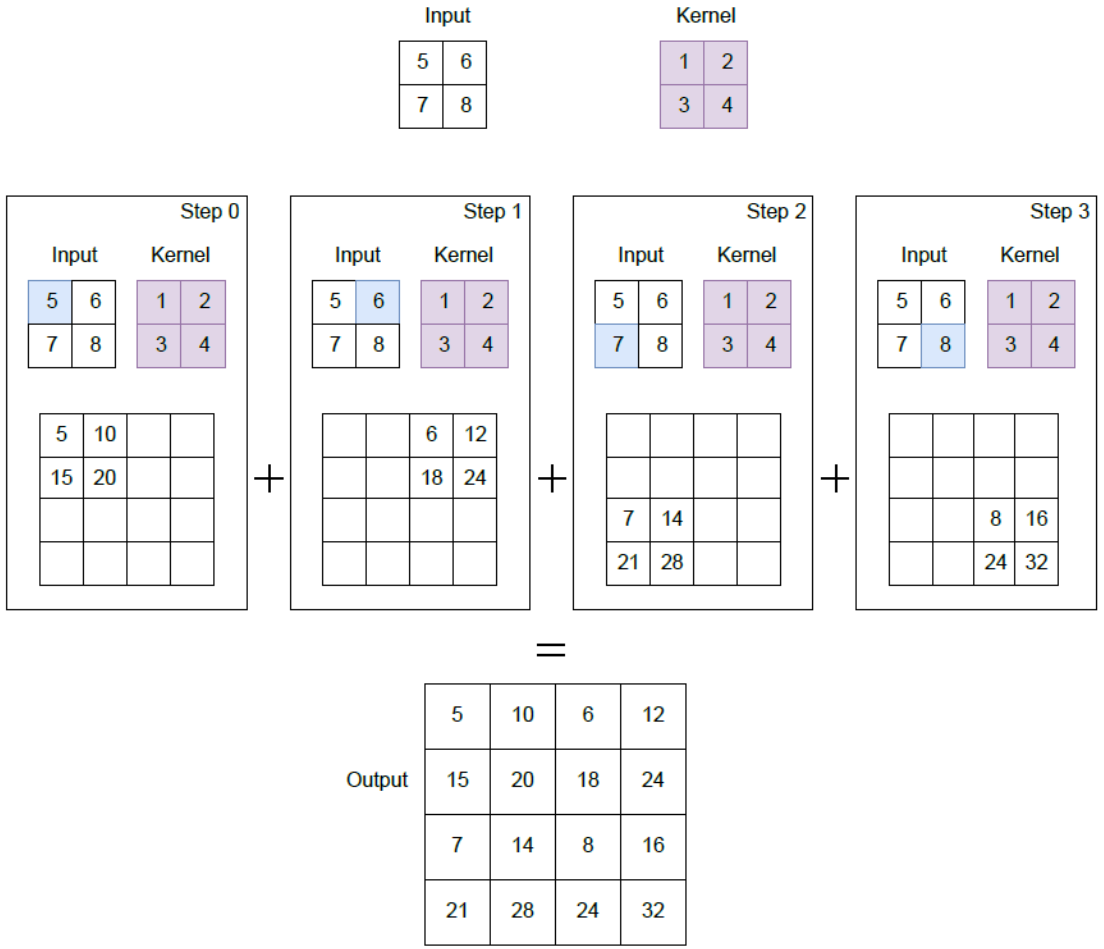
2D transpose convolution with stride 1



Source: Math and Architectures of Deep Learning, Krishenedu Chauhury

Transposed convolution: 2D

2D transpose convolution with stride 2



Source: Math and Architectures of Deep Learning, Krishenedu Chauhury

Output size of transposed convolution

- n x n image
- k x k filter
- stride s
- No padding

Output size for transposed convolution:

$$o = (n - 1) \cdot s + k$$

Output size for convolution:

$$o = \frac{n - k}{s} + 1$$